



CSE 711 - Compilers

Lecture 3 (Part 3)

Syntax Analysis

Ms. Samia Muntaha

Lecturer,

Department of Computer Science and Engineering
University of Chittagong

Outline

- ▶ Bottom-Up Parsing
- ▶ Shift Reduce Parsing
- ▶ LR Parsing
- ▶ SLR Parsing
- ▶ CLR & LALR parsing

Bottom-Up Parsing

- A **bottom-up parser** creates the parse tree of the given input starting from leaves towards the root
- A bottom-up parser tries to find the **right-most derivation** of the given input in the reverse order.

$S \Rightarrow \dots \Rightarrow \omega$ (the right-most derivation of ω)

$\leftarrow$ (the bottom-up parser finds the right-most derivation in the reverse order)

Bottom Up Parsing

- ▶ LR Parsing
 - ▶ Also called “Shift-Reduce Parsing”
- ▶ Find a rightmost derivation
- ▶ Finds it in reverse order
- ▶ LR Grammars
 - ▶ Can be parsed with an LR Parser
- ▶ LR Languages
 - ▶ Can be described with LR Grammar
 - ▶ Can be parsed with an LR Parser

LR Parsing Techniques

- ▶ **LR Parsing**

- ▶ Most General Approach

- ▶ **SLR**

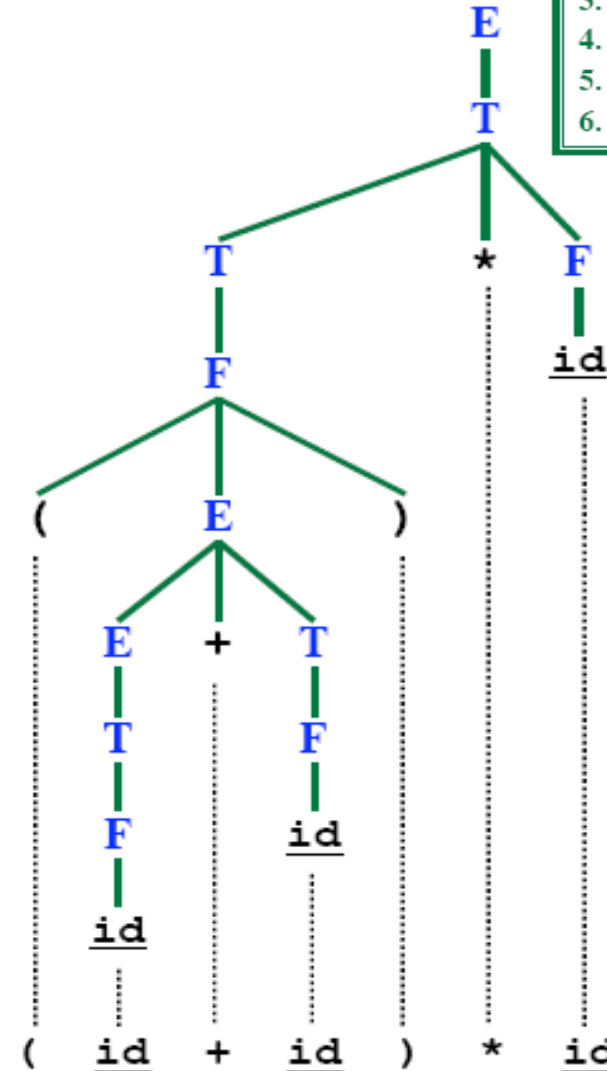
- ▶ Simpler algorithm, but not as general

- ▶ **LALR & CLR**

- ▶ More complex, but saves space

LL vs. LR

- ▶ LR (shift reduce) is more powerful than LL (predictive parsing)
- ▶ Can detect a syntactic error as soon as possible.
- ▶ LR is difficult to do by hand (unlike LL) and
- ▶ LL accepts a much smaller set of grammars.

$$F \rightarrow \text{id}$$
$$(\underline{\text{id}} + \underline{\text{id}}) * \underline{\text{id}}$$


1. $E \rightarrow E + T$
2. $E \rightarrow T$
3. $T \rightarrow T * F$
4. $T \rightarrow F$
5. $F \rightarrow (E)$
6. $F \rightarrow \underline{\text{id}}$

Rightmost Derivation In reverse

Rules Used:

$F \rightarrow \underline{id}$

$T \rightarrow F$

$E \rightarrow T$

$F \rightarrow \underline{id}$

$T \rightarrow F$

$E \rightarrow E + T$

Right-Sentential Forms:

$(\underline{id} + \underline{id}) * \underline{id}$

$(F + \underline{id}) * \underline{id}$

$(T + \underline{id}) * \underline{id}$

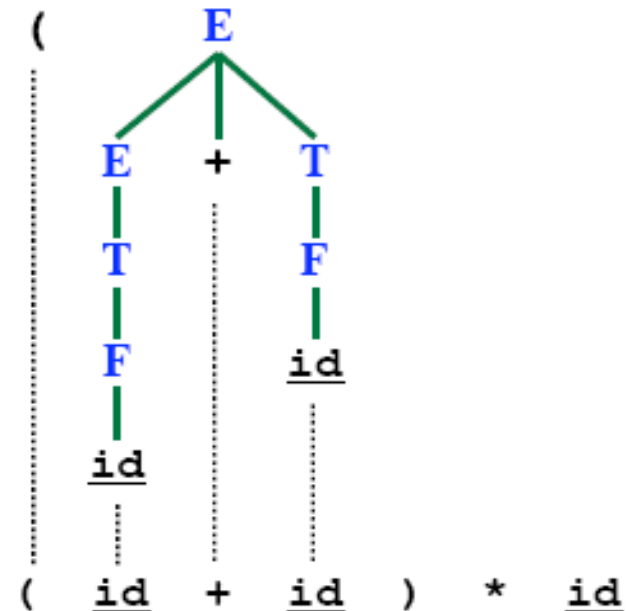
$(E + \underline{id}) * \underline{id}$

$(E + F) * \underline{id}$

$(E + T) * \underline{id}$

$(E) * \underline{id}$

1. $E \rightarrow E + T$
2. $E \rightarrow T$
3. $T \rightarrow T * F$
4. $T \rightarrow F$
5. $F \rightarrow (E)$
6. $F \rightarrow \underline{id}$



Rightmost Derivation In reverse

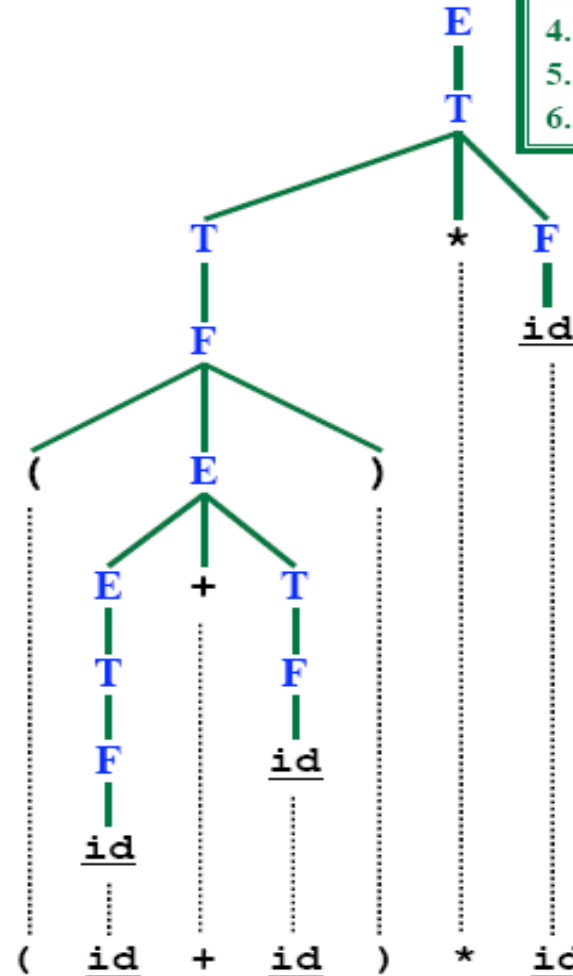
Rules Used:

$F \rightarrow \underline{id}$
 $T \rightarrow F$
 $E \rightarrow T$
 $F \rightarrow \underline{id}$
 $T \rightarrow F$
 $E \rightarrow E + T$
 $F \rightarrow (E)$
 $T \rightarrow F$
 $F \rightarrow \underline{id}$
 $T \rightarrow T * F$
 $E \rightarrow T$

Right-Sentential Forms:

$(\underline{id} + \underline{id}) * \underline{id}$
 $(F + \underline{id}) * \underline{id}$
 $(T + \underline{id}) * \underline{id}$
 $(E + \underline{id}) * \underline{id}$
 $(E + F) * \underline{id}$
 $(E + T) * \underline{id}$
 $(E) * \underline{id}$
 $F * \underline{id}$
 $T * \underline{id}$
 $T * F$
 T
 E

1. $E \rightarrow E + T$
2. $E \rightarrow T$
3. $T \rightarrow T * F$
4. $T \rightarrow F$
5. $F \rightarrow (E)$
6. $F \rightarrow \underline{id}$



LR parsing corresponds to rightmost derivation in reverse

Rightmost Derivation In reverse

Reduction

- A reduction step replaces a specific substring (matching the body of a production)

(id + id) * id
(F + id) * id
(T + id) * id
(E + id) * id
(E + F) * id
(E + T) * id

(E) * id
F * id
T * id
T * F
T
E

1.	$E \rightarrow E + T$
2.	$E \rightarrow T$
3.	$T \rightarrow T * F$
4.	$T \rightarrow F$
5.	$F \rightarrow (E)$
6.	$F \rightarrow \underline{id}$

- Reduction is the opposite of derivation
- Bottom up parsing is a process of **reducing** a string ω to the start symbol S of the grammar

Shift-Reduce Parsing

- Bottom-up parsing is also known as **shift-reduce parsing** because its two main actions are shift and reduce.
- data structures: input-string and stack
- Operations
 - At each **shift** action, the current symbol in the input string is pushed to a stack.
 - At each **reduction** step, the symbols at the top of the stack (this symbol sequence is the right side of a production) will be replaced by the non-terminal at the left side of that production.
 - **Accept**: Announce successful completion of parsing
 - **Error**: Discover a syntax error and call error recovery

Shift Reduce Parsing Example

$S \rightarrow a T R e$
 $T \rightarrow T b c \mid b$
 $R \rightarrow d$

Remaining input: **a**bbcde

Rightmost derivation:

$S \rightarrow a T R e$
 $\rightarrow a T d e$
 $\rightarrow a T b c d e$
 $\rightarrow a b b c d e$

Shift Reduce Parsing

$S \rightarrow a T R e$
 $T \rightarrow T b c \mid b$
 $R \rightarrow d$

➔ Shift a

Remaining input: **b**bcde

a

Rightmost derivation:

$S \rightarrow a T R e$
 $\rightarrow a T d e$
 $\rightarrow a T b c d e$
 $\rightarrow \underline{a} b b c d e$

Shift Reduce Parsing

$S \rightarrow a T R e$
 $T \rightarrow T b c \mid b$
 $R \rightarrow d$

Remaining input: **b**cde

➔ Shift a, Shift b

a b

Rightmost derivation:

$S \rightarrow a T R e$
 $\rightarrow a T d e$
 $\rightarrow a T b c d e$
 $\rightarrow \underline{a b} b c d e$

Shift Reduce Parsing

$S \rightarrow a T R e$

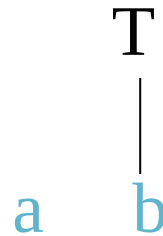
$T \rightarrow T b c \mid b$

$R \rightarrow d$

Remaining input: **b**cde

➔ Shift a, Shift b

➔ Reduce $T \rightarrow b$



Rightmost derivation:

$S \rightarrow a T R e$

➔ $a T d e$

➔ **a T b c d e**

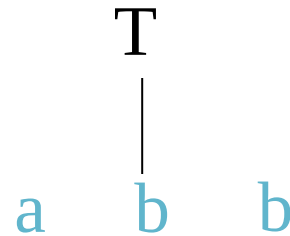
➔ **a b b c d e**

Shift Reduce Parsing

$S \rightarrow a T R e$
 $T \rightarrow T b c \mid b$
 $R \rightarrow d$

➔ Shift a, Shift b
➔ Reduce $T \rightarrow b$
➔ Shift b

Remaining input: **c**de



Rightmost derivation:

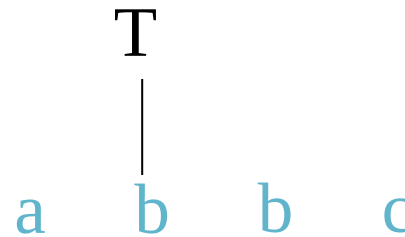
$S \rightarrow a T R e$
 $\rightarrow a T d e$
➔ $a T b$ c d e
➔ **a b b c d e**

Shift Reduce Parsing

$S \rightarrow a T R e$
 $T \rightarrow T b c \mid b$
 $R \rightarrow d$

➔ Shift a, Shift b
➔ Reduce $T \rightarrow b$
➔ Shift b, Shift c

Remaining input: **de**



Rightmost derivation:

$S \rightarrow a T R e$
 $\rightarrow a T d e$
➔ $a T b c d e$
➔ **$a b b c d e$**

Shift Reduce Parsing

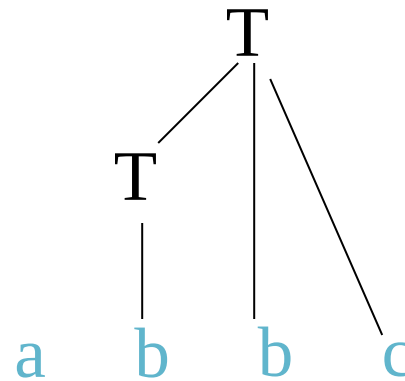
$S \rightarrow a T R e$

$T \rightarrow T b c \mid b$

$R \rightarrow d$

Remaining input: **de**

- ➔ Shift a, Shift b
- ➔ Reduce $T \rightarrow b$
- ➔ Shift b, Shift c
- ➔ Reduce $T \rightarrow T b c$



Rightmost derivation:

$S \rightarrow a T R e$

➔ a **T** **d** **e**

➔ **a** **T** **b** **c** **d** **e**

➔ **a** **b** **b** **c** **d** **e**

Shift Reduce Parsing

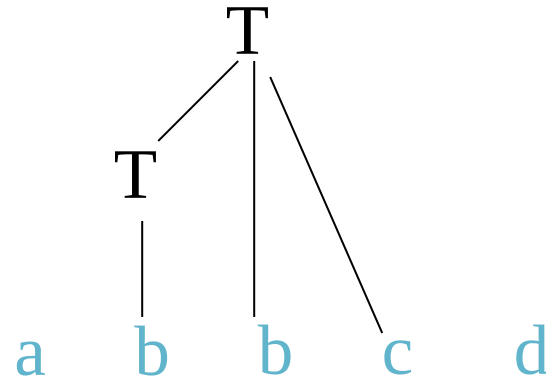
$S \rightarrow a T R e$

$T \rightarrow T b c \mid b$

$R \rightarrow d$

Remaining input: **e**

- ➔ Shift a, Shift b
- ➔ Reduce $T \rightarrow b$
- ➔ Shift b, Shift c
- ➔ Reduce $T \rightarrow T b c$
- ➔ Shift d



Rightmost derivation:

$S \rightarrow a T R e$

➔ $a T d e$

➔ $a T b c d e$

➔ $a b b c d e$

Shift Reduce Parsing

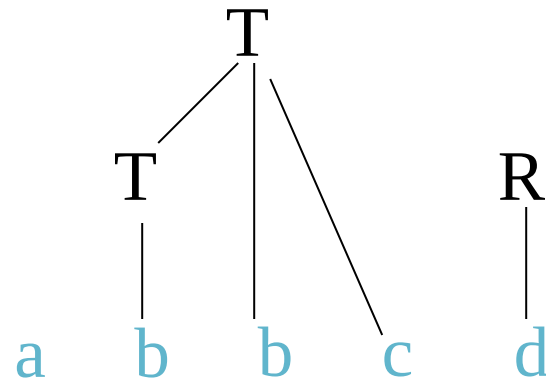
$S \rightarrow a T R e$

$T \rightarrow T b c \mid b$

$R \rightarrow d$

Remaining input: **e**

- ➔ Shift a, Shift b
- ➔ Reduce $T \rightarrow b$
- ➔ Shift b, Shift c
- ➔ Reduce $T \rightarrow T b c$
- ➔ Shift d
- ➔ Reduce $R \rightarrow d$



Rightmost derivation:

$S \rightarrow \underline{a} T R e$
 $\rightarrow a T \underline{d} e$
 $\rightarrow a T b c d e$
 $\rightarrow a b b c d e$

Shift Reduce Parsing

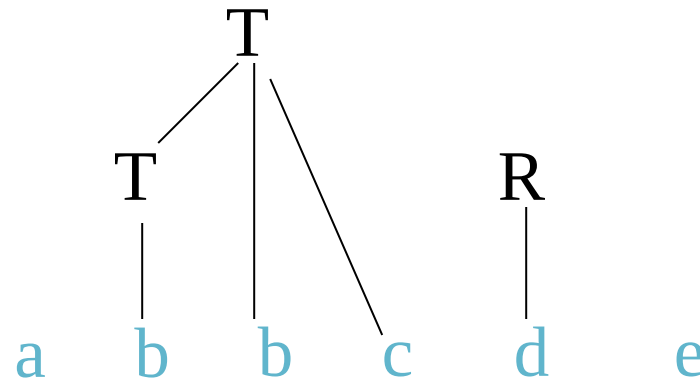
$S \rightarrow a T R e$

$T \rightarrow T b c \mid b$

$R \rightarrow d$

- ➔ Shift a, Shift b
- ➔ Reduce $T \rightarrow b$
- ➔ Shift b, Shift c
- ➔ Reduce $T \rightarrow T b c$
- ➔ Shift d
- ➔ Reduce $R \rightarrow d$
- ➔ Shift e

Remaining input:



Rightmost derivation:

$S \rightarrow \underline{a T R e}$
 $\rightarrow a T d e$
 $\rightarrow a T b c d e$
 $\rightarrow a b b c d e$

Shift Reduce Parsing

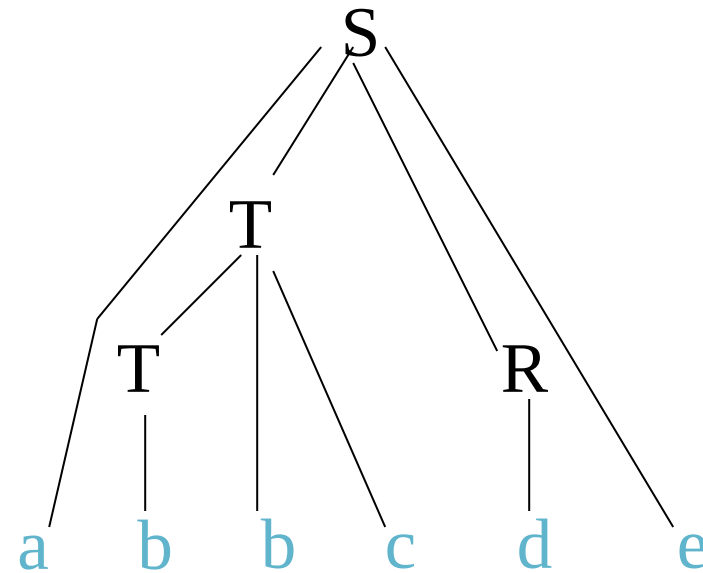
$S \rightarrow a T R e$

$T \rightarrow T b c \mid b$

$R \rightarrow d$

- ➔ Shift a, Shift b
- ➔ Reduce $T \rightarrow b$
- ➔ Shift b, Shift c
- ➔ Reduce $T \rightarrow T b c$
- ➔ Shift d
- ➔ Reduce $R \rightarrow d$
- ➔ Shift e
- ➔ Reduce $S \rightarrow a T R e$

Remaining input:



Rightmost derivation:

$S \rightarrow a T R e$

$\rightarrow a T d e$

$\rightarrow a T b c d e$

$\rightarrow a b b c d e$

Example Shift-Reduce Parsing

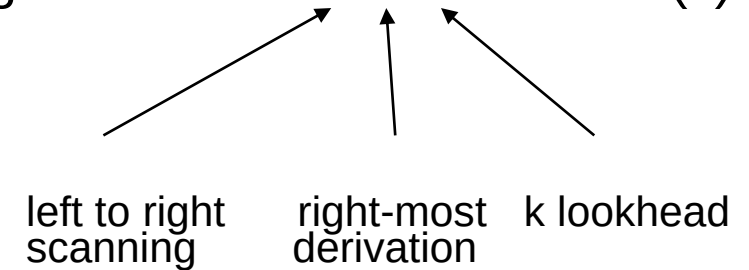
Consider the grammar:

Stack	Input	Action
\$	id ₁ + id ₂ \$	shift
\$id ₁	+ id ₂ \$	reduce 6
\$F	+ id ₂ \$	reduce 4
\$T	+ id ₂ \$	reduce 2
\$E	+ id ₂ \$	shift
\$E +	id ₂ \$	shift
\$E + id ₂		reduce 6
\$E + F		reduce 4
\$E + T		reduce 1
\$E		accept

1.	$E \rightarrow E + T$
2.	$E \rightarrow T$
3.	$T \rightarrow T * F$
4.	$T \rightarrow F$
5.	$F \rightarrow (E)$
6.	$F \rightarrow \underline{id}$

Conflicts During Shift-Reduce Parsing

- ▶ There are context-free grammars for which shift-reduce parsers cannot be used.
- ▶ Stack contents and the next input symbol may not decide action:
 - ▶ **shift/reduce conflict**: Whether make a shift operation or a reduction.
 - ▶ **reduce/reduce conflict**: The parser cannot decide which of several reductions to make.
- ▶ If a shift-reduce parser cannot be used for a grammar, that grammar is called as non-LR(k) grammar.



- ▶ An ambiguous grammar can never be a LR grammar.

Shift-Reduce Conflict in Ambiguous Grammar

stmt → **if** *expr* **then** *stmt*
 | **if** *expr* **then** *stmt* **else** *stmt*
 | **other**

STACK

....**if** *expr* **then** *stmt*

INPUT

else\$

- ▶ We can't decide whether to shift or reduce?

Reduce-Reduce Conflict in Ambiguous Grammar

1. $stmt \rightarrow \mathbf{id}(parameter_list)$
2. $stmt \rightarrow expr:=expr$
3. $parameter_list \rightarrow parameter_list, parameter$
4. $parameter_list \rightarrow parameter$
5. $parameter_list \rightarrow \mathbf{id}$
6. $expr \rightarrow \mathbf{id}(expr_list)$
7. $expr \rightarrow \mathbf{id}$
8. $expr_list \rightarrow expr_list, expr$
9. $expr_list \rightarrow expr$

STACK

....**id** (**id**

INPUT

, **id**) ...\$

- We can't decide which production will be used to reduce **id**?

Shift-Reduce Parsers

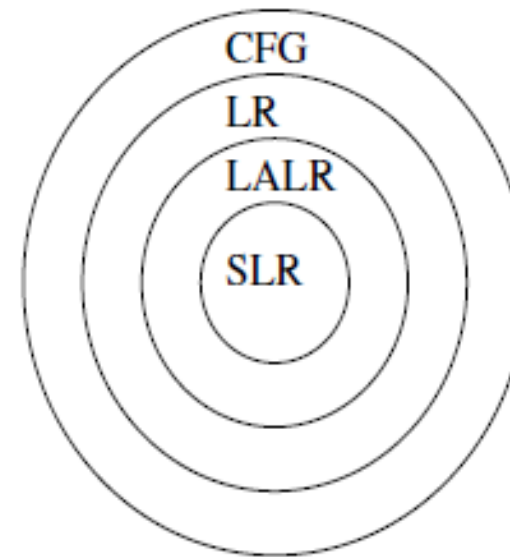
There are two main categories of shift-reduce parsers

1. Operator-Precedence Parser

- simple, but only a small class of grammars.

2. LR-Parsers

- covers wide range of grammars.
 - SLR – simple LR parser
 - LR – most general LR parser
 - LALR – intermediate LR parser (lookahead LR parser)
- SLR, LR and LALR work same, only their parsing tables are different.



LR Parsers

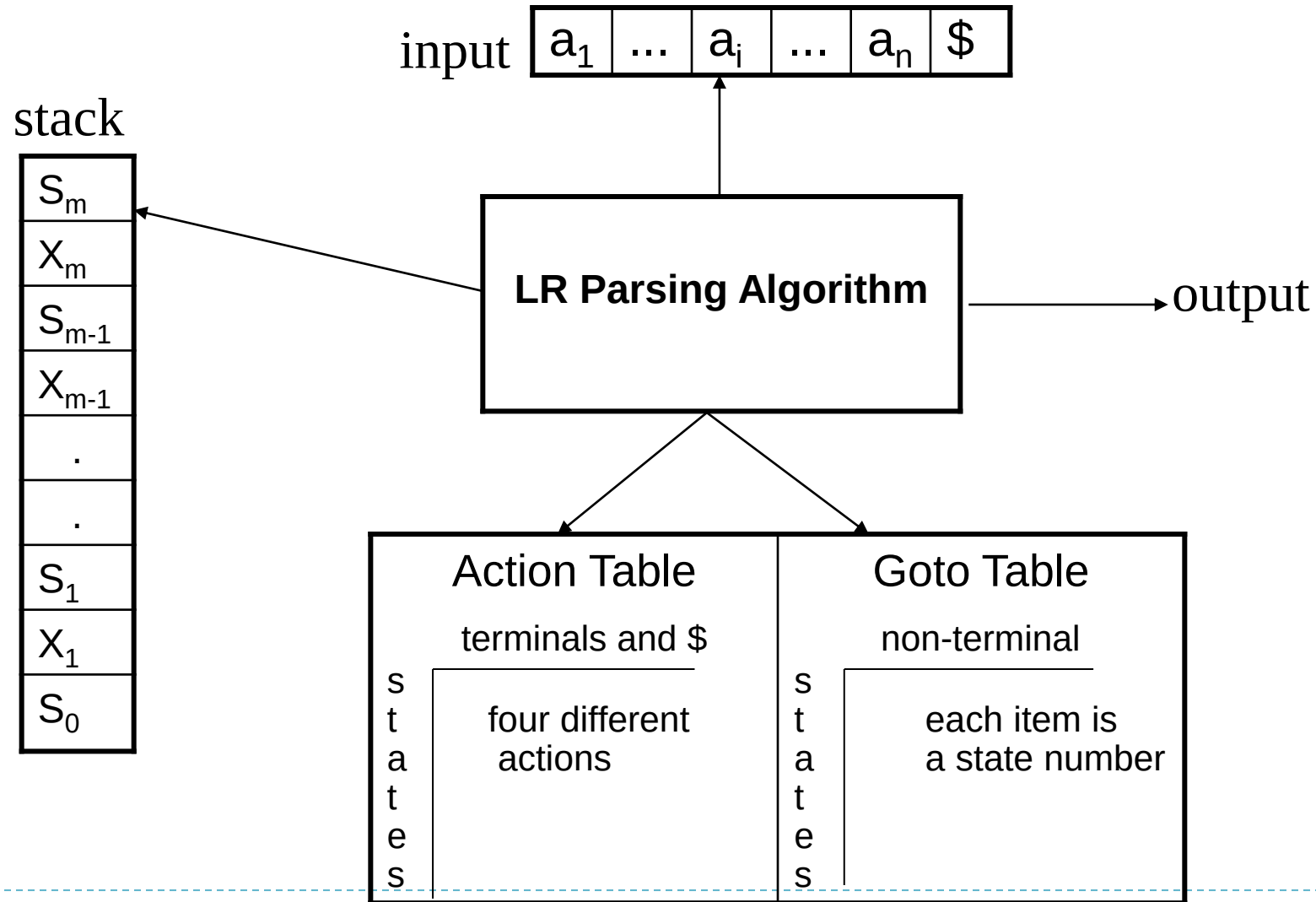
LR parsing is attractive because:

- LR parsing is most general non-backtracking shift-reduce parsing, yet it is still efficient.
- The class of grammars that can be parsed using LR methods is a proper superset of the class of grammars that can be parsed with predictive parsers.
 - LL(1)-Grammars $\subset$ LR(1)-Grammars
- An LR-parser can detect a syntactic error as soon as it is possible to do so a left-to-right scan of the input.
- LR parsers can be constructed to recognize virtually all programming language constructs for which CFG grammars can be written

Drawback of LR method:

- Too much work to construct LR parser by hand
 - Fortunately tools (LR parsers generators) are available

LR Parsing Algorithm



-

Constructing SLR Parsing Tables – LR(0) Item

- An item indicates how much of a production we have seen at a given point in the parsing process
- For Example the item $A \rightarrow X \bullet YZ$
 - We have already seen on the input a string derivable from X
 - We hope to see a string derivable from YZ
- For Example the item $A \rightarrow \bullet XYZ$
 - We hope to see a string derivable from XYZ
- For Example the item $A \rightarrow XYZ \bullet$
 - We have already seen on the input a string derivable from XYZ
 - It is possibly time to reduce XYZ to A
- **Special Case:**
Rule: $A \rightarrow \epsilon$ yields only one item
 $A \rightarrow \bullet$



Constructing SLR Parsing Tables

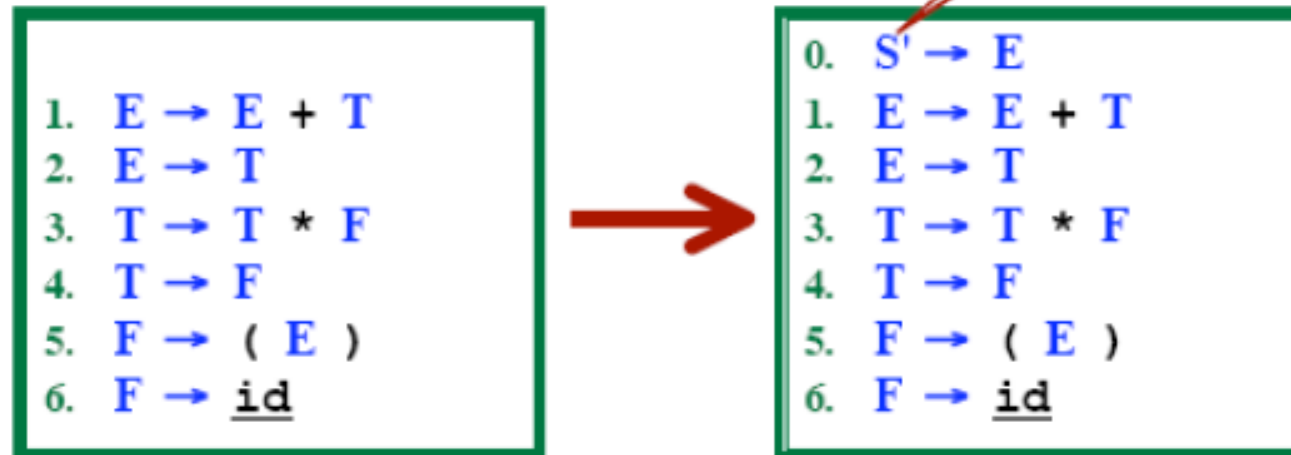
- A collection of sets of LR(0) items (**the canonical LR(0) collection**) is the basis for constructing SLR parsers.
- Canonical LR(0) collection provides the basis of constructing a DFA called **LR(0) automaton**
 - This DFA is used to make parsing decisions
- Each state of LR(0) automaton represents a set of items in the canonical LR(0) collection
- To construct the canonical LR(0) collection for a grammar
 - Augmented Grammar
 - CLOSURE function
 - GOTO function



Grammar Augmentation

Augment the grammar by adding...

- A new start symbol, S'
- A new rule $S' \rightarrow S$



Our goal is to find an S' , followed by $\$$.

$S' \rightarrow \bullet E, \$$

Whenever we are about to reduce using rule 0...

Accept! Parse is finished!

The Closure Operation

- If I is a set of LR(0) items for a grammar G , then ***closure(I)*** is the set of LR(0) items constructed from I by the two rules:
 1. Initially, every LR(0) item in I is added to ***closure(I)***.
 2. If $A \rightarrow \alpha.B\beta$ is in ***closure(I)*** and $B \rightarrow \gamma$ is a production rule of G ;
then $B \rightarrow .\gamma$ will be in the ***closure(I)***.
We will apply this rule until no more new LR(0) items can be added to ***closure(I)***.

The Closure Operation -- Example

$E' \rightarrow E$

$E \rightarrow E+T$

$E \rightarrow T$

$T \rightarrow T^*F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow id$

$\text{closure}(\{E' \rightarrow \blacksquare E\}) =$

$\{ E' \rightarrow \bullet E \leftarrow \text{kernel items}$

$E \rightarrow \bullet E+T$

$E \rightarrow \bullet T$

$T \rightarrow \bullet T^*F$

$T \rightarrow \bullet F$

$F \rightarrow \bullet (E)$

$F \rightarrow \bullet id \}$

GOTO Operation

- If I is a set of LR(0) items and X is a grammar symbol (terminal or non-terminal), then $\text{GOTO}(I, X)$ is defined as follows:
 - If $A \rightarrow \alpha \cdot X \beta$ in I
then every item in $\text{closure}(\{A \rightarrow \alpha X \cdot \beta\})$ will be in $\text{GOTO}(I, X)$.

Example:

$I = \{ E' \rightarrow \cdot E, E \rightarrow \cdot E + T, E \rightarrow \cdot T, \\ T \rightarrow \cdot T * F, T \rightarrow \cdot F, \\ F \rightarrow \cdot (E), F \rightarrow \cdot \text{id} \}$

$\text{GOTO}(I, E) = \{ E' \rightarrow E \cdot, E \rightarrow E \cdot + T \}$

$\text{GOTO}(I, T) = \{ E \rightarrow T \cdot, T \rightarrow T \cdot * F \}$

$\text{GOTO}(I, F) = \{ T \rightarrow F \cdot \}$

$\text{GOTO}(I, () = \{ F \rightarrow (\cdot E), E \rightarrow \cdot E + T, E \rightarrow \cdot T, T \rightarrow \cdot T * F, T \rightarrow \cdot F, \\ F \rightarrow \cdot (E), F \rightarrow \cdot \text{id} \}$

$\text{GOTO}(I, \text{id}) = \{ F \rightarrow \text{id} \cdot \}$

LR(0) Automation

- ❑ Start with **start rule** & compute **initial state with closure**
- ❑ Pick one of the items from the states and **move “.” to the right one symbol** (as if you parsed the symbol)
 - this creates a **new item..**
 - ... and a **new state** when you **compute the closure of the new item**
 - **mark the edge between the two states** with:
 - ✓ a terminal T, if you moved “.” over T
 - ✓ a non-terminal X, if you moved “.” over x
- ❑ **Continue until there are no further ways to move “.” across items and generate the new states or new edges in the automation.**



Canonical Collection of LR(0) Items

Grammar:

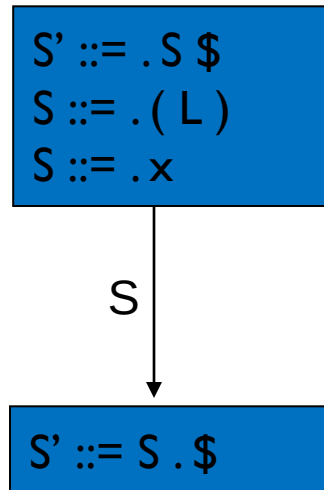
- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L , S$

$S' ::= . S \$$
 $S ::= . (L)$
 $S ::= . x$

Canonical Collection of LR(0) Items

Grammar:

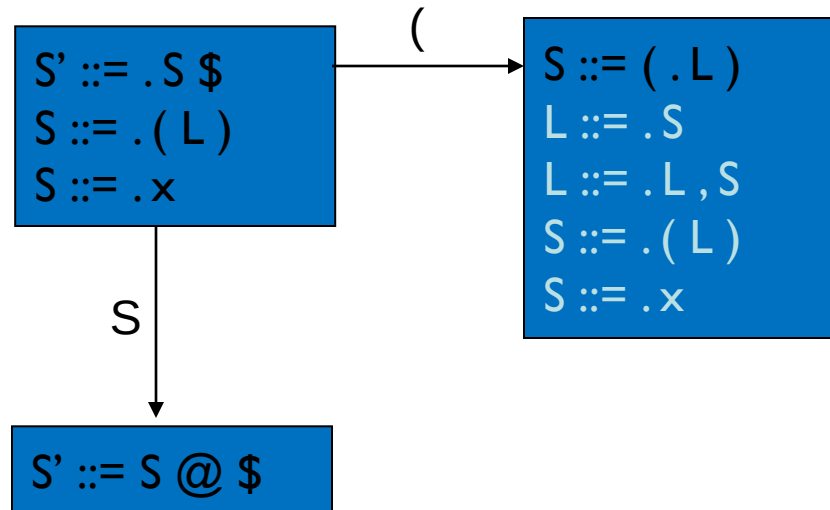
- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L , S$



Canonical Collection of LR(0) Items

Grammar:

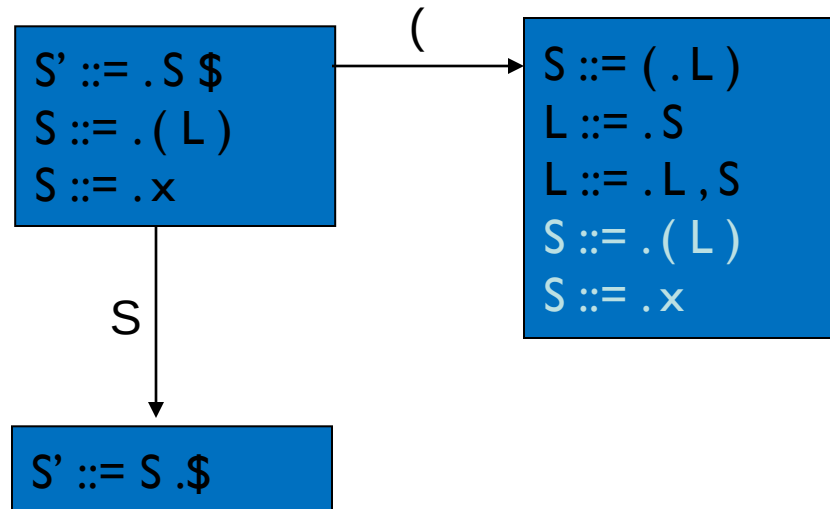
- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L , S$



Canonical Collection of LR(0) Items

Grammar:

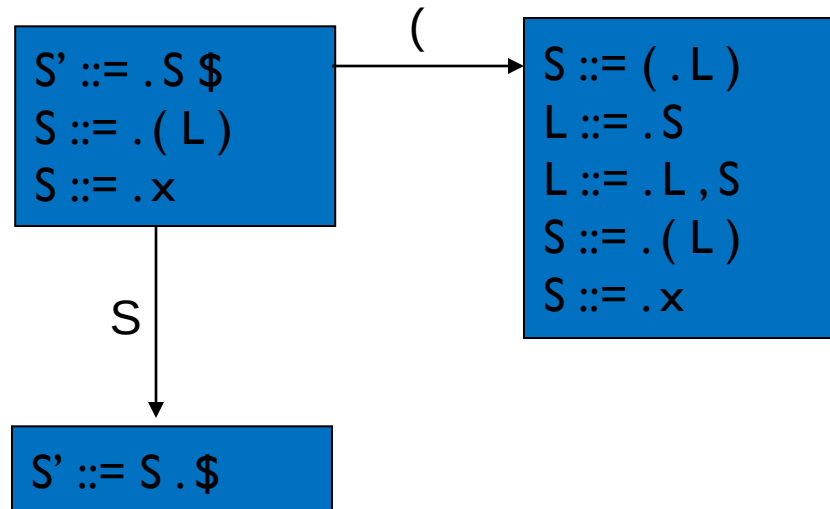
- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L , S$



Canonical Collection of LR(0) Items

Grammar:

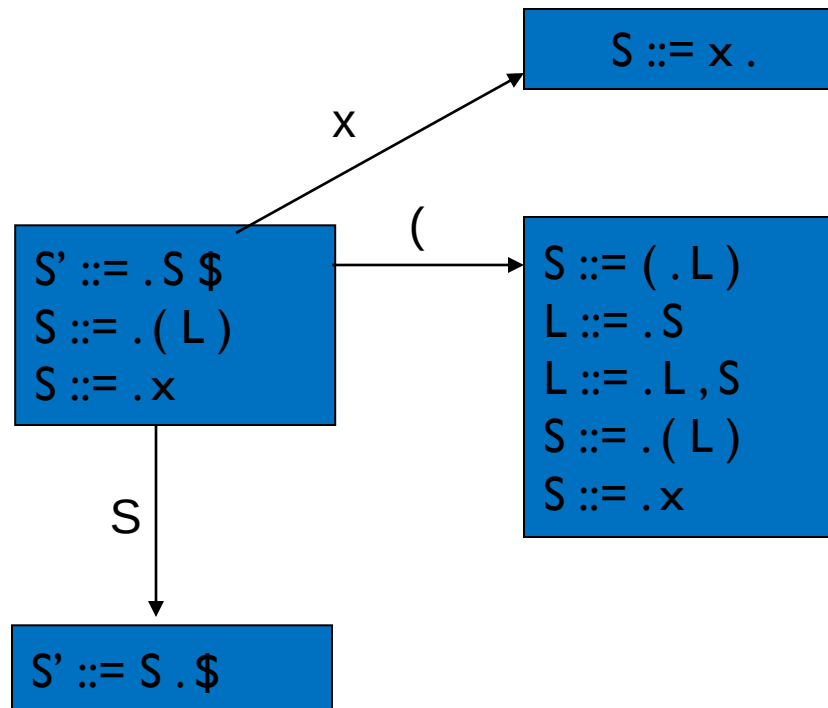
- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L , S$



Canonical Collection of LR(0) Items

Grammar:

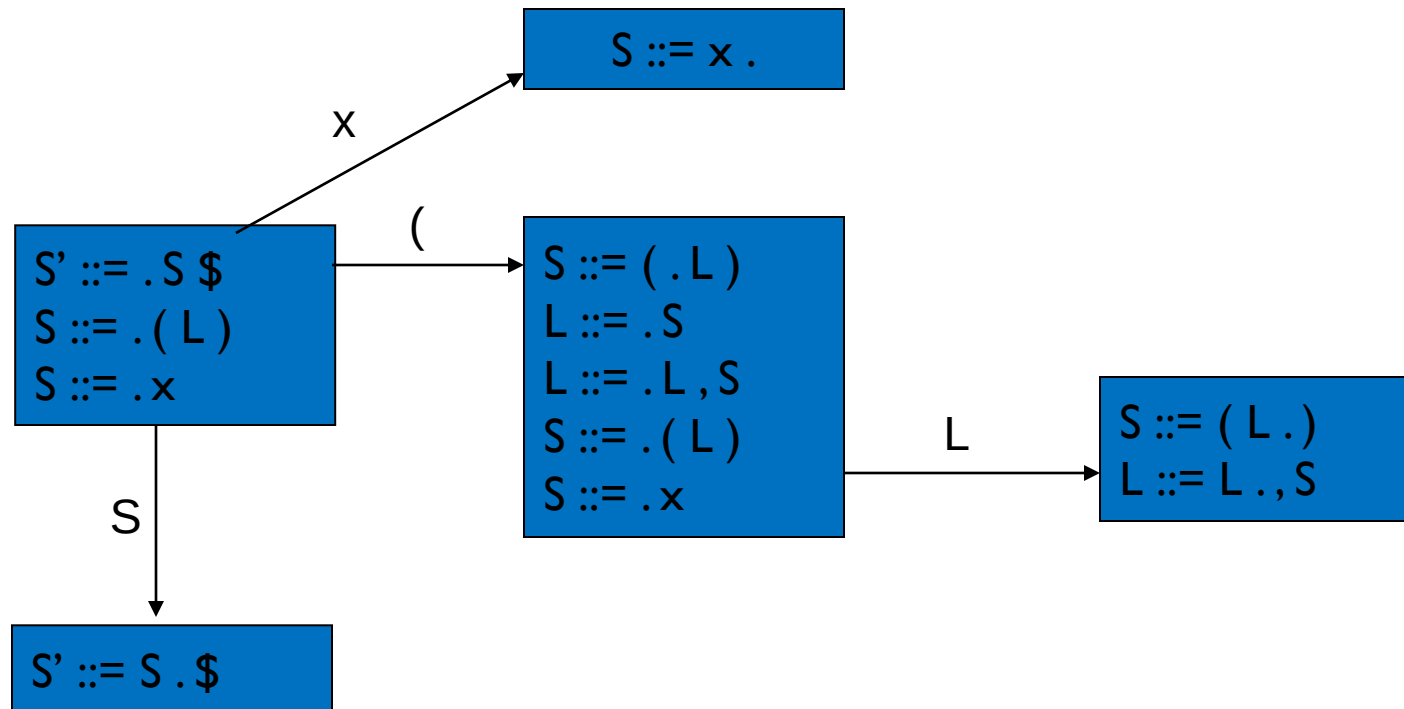
- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L , S$



Canonical Collection of LR(0) Items

Grammar:

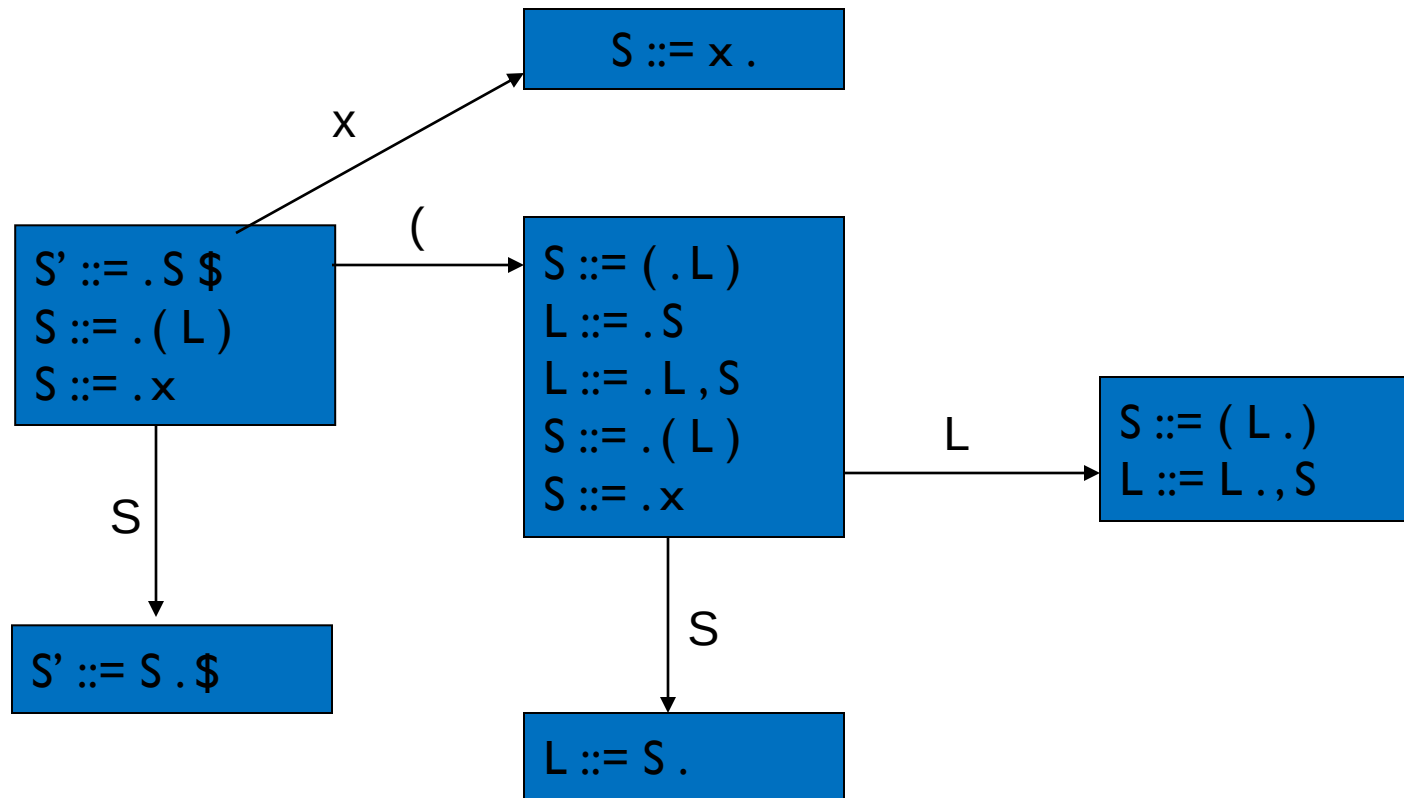
- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L , S$



Canonical Collection of LR(0) Items

Grammar:

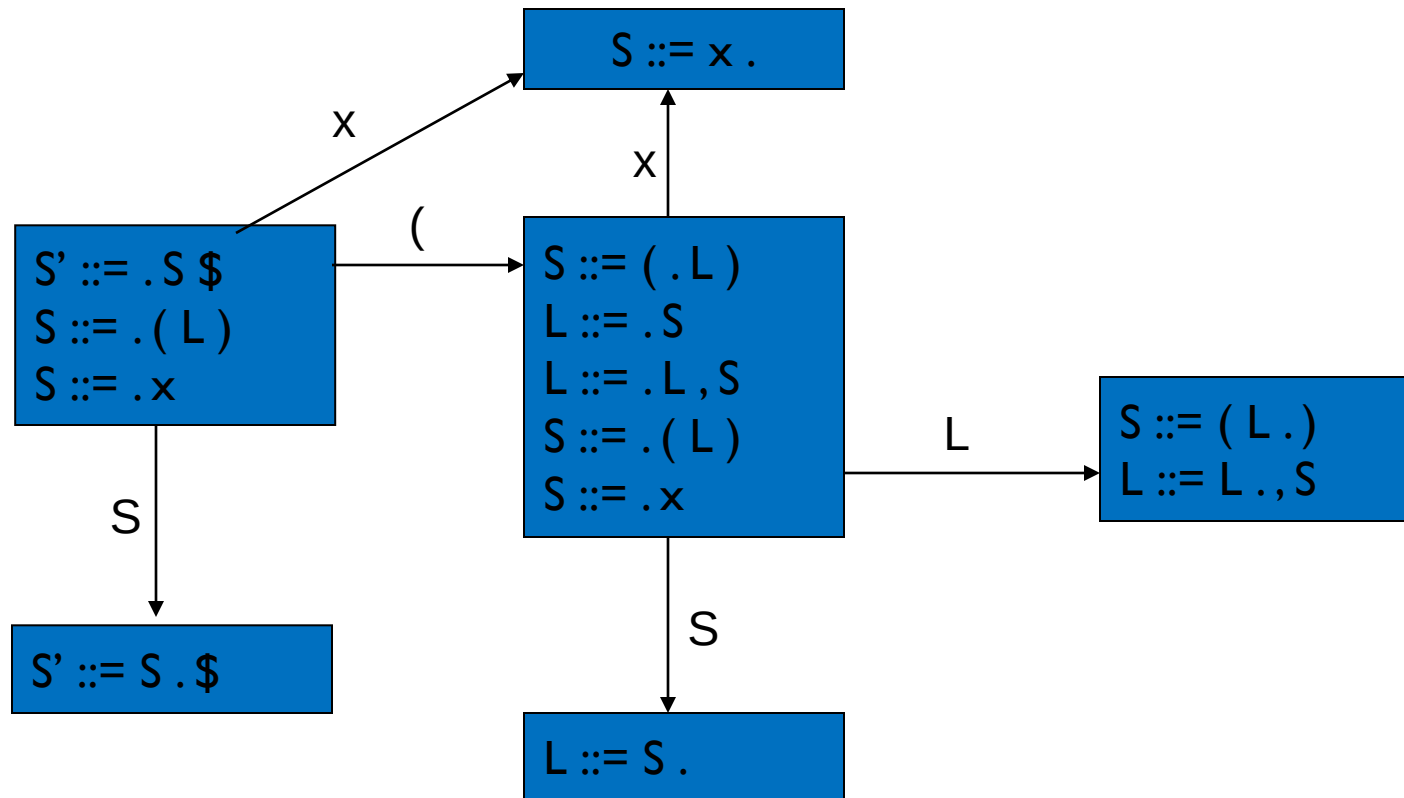
- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L , S$



Canonical Collection of LR(0) Items

Grammar:

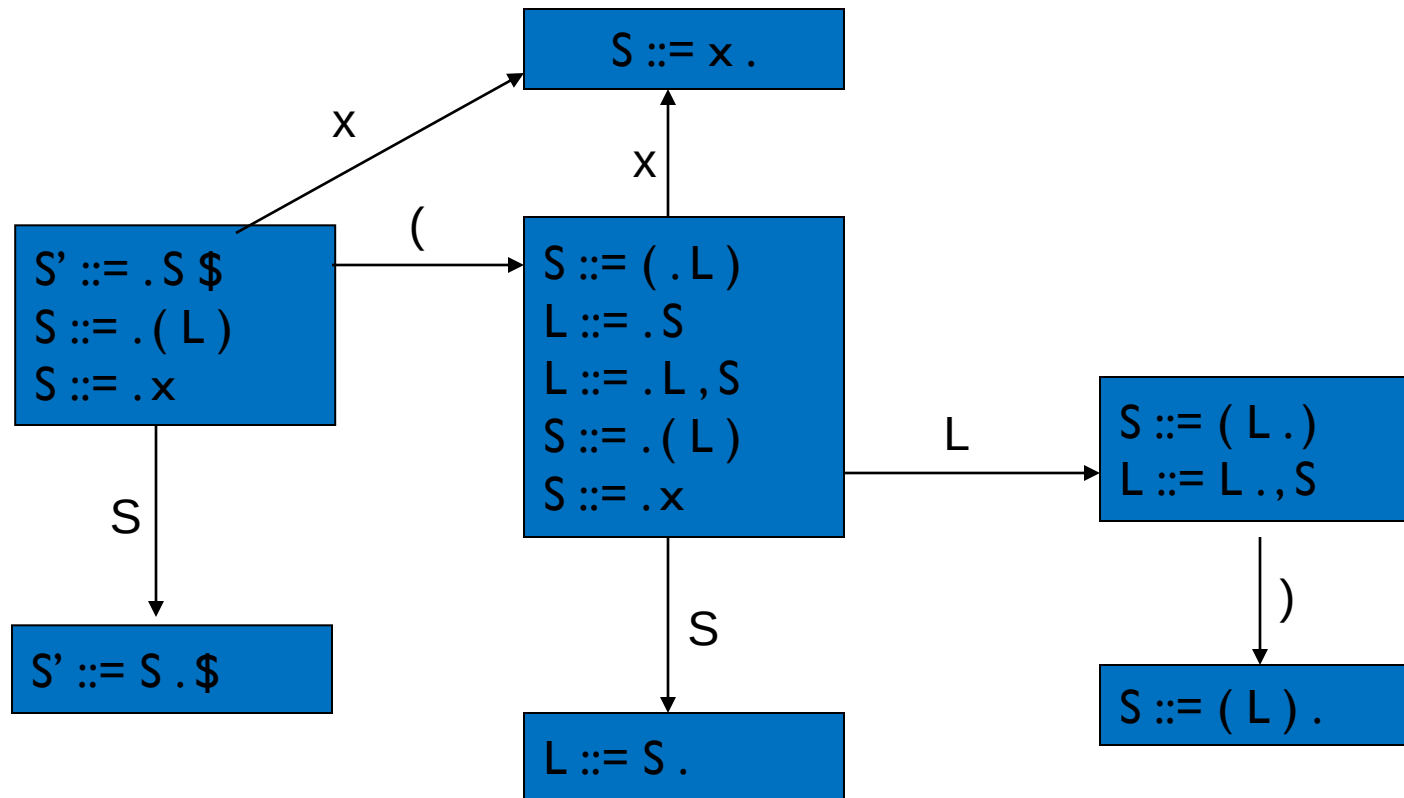
- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L , S$



Canonical Collection of LR(0) Items

Grammar:

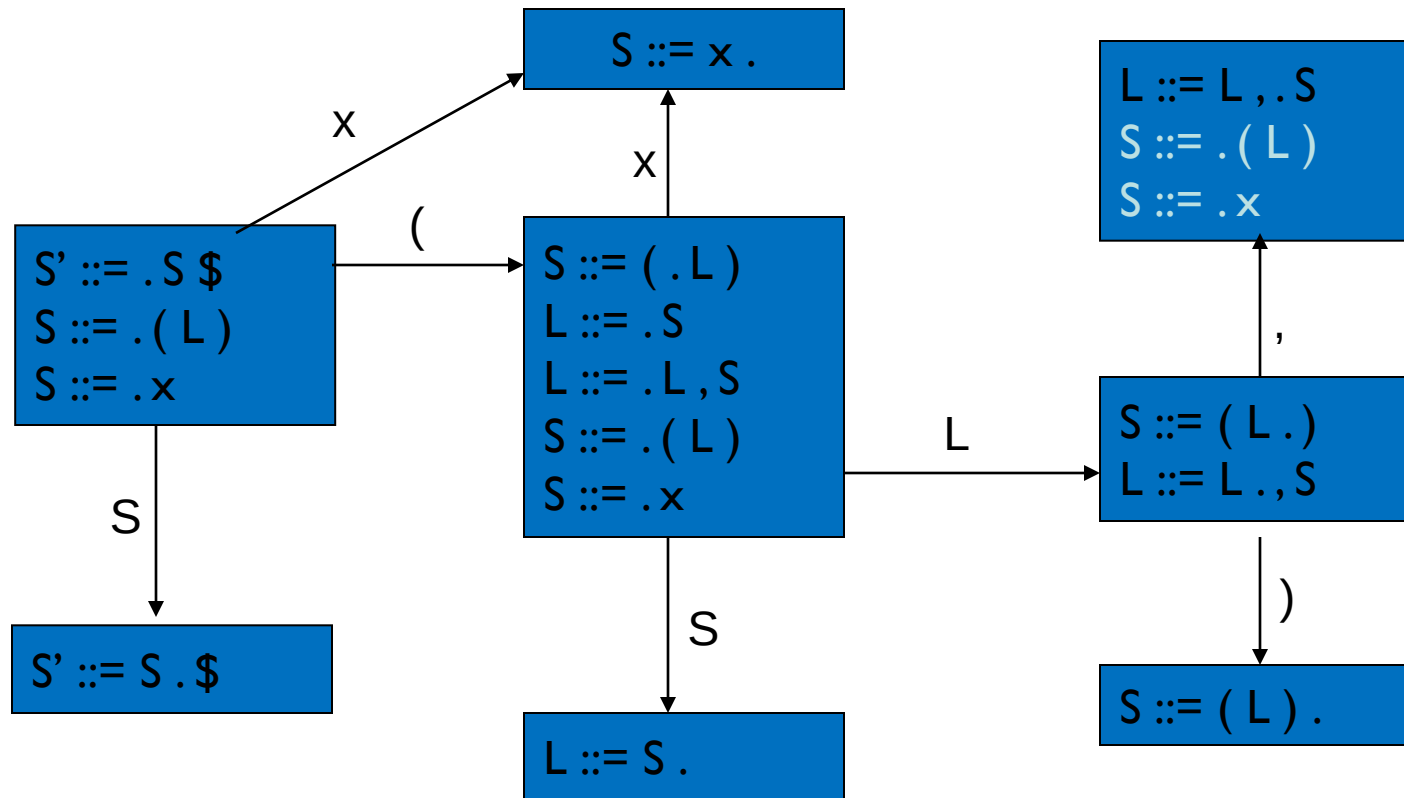
- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L , S$



Canonical Collection of LR(0) Items

Grammar:

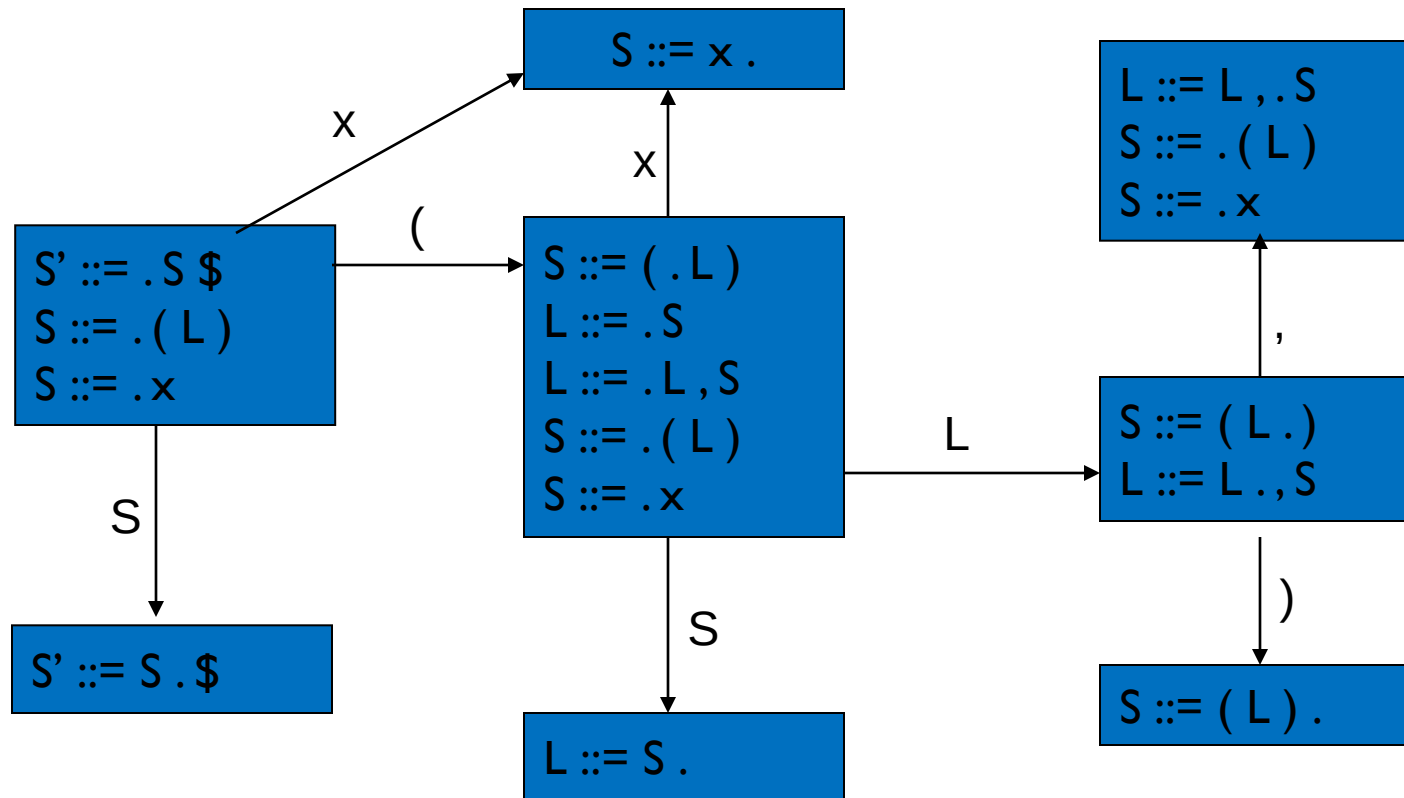
- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L , S$



Canonical Collection of LR(0) Items

Grammar:

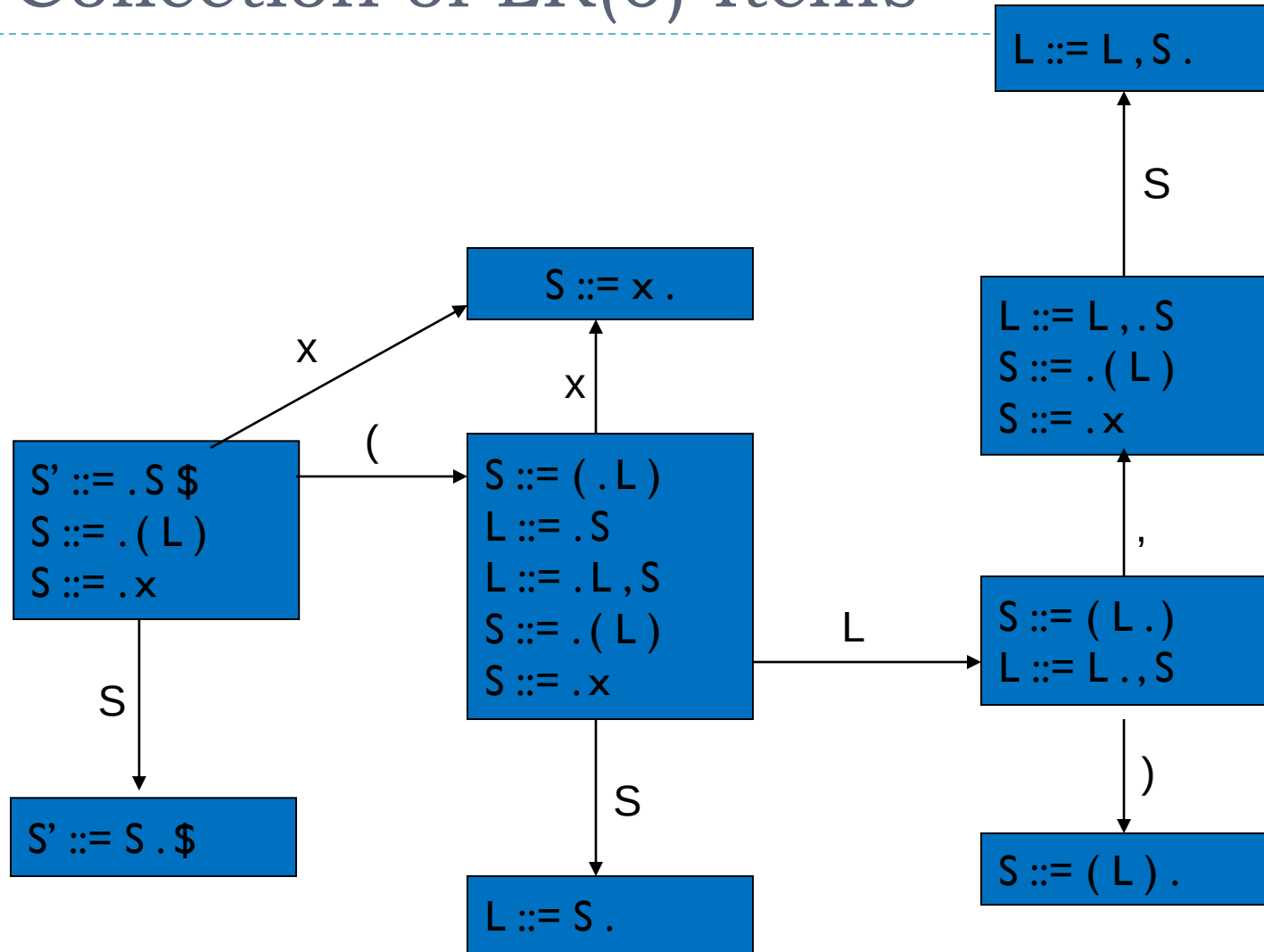
- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L , S$



Canonical Collection of LR(0) Items

Grammar:

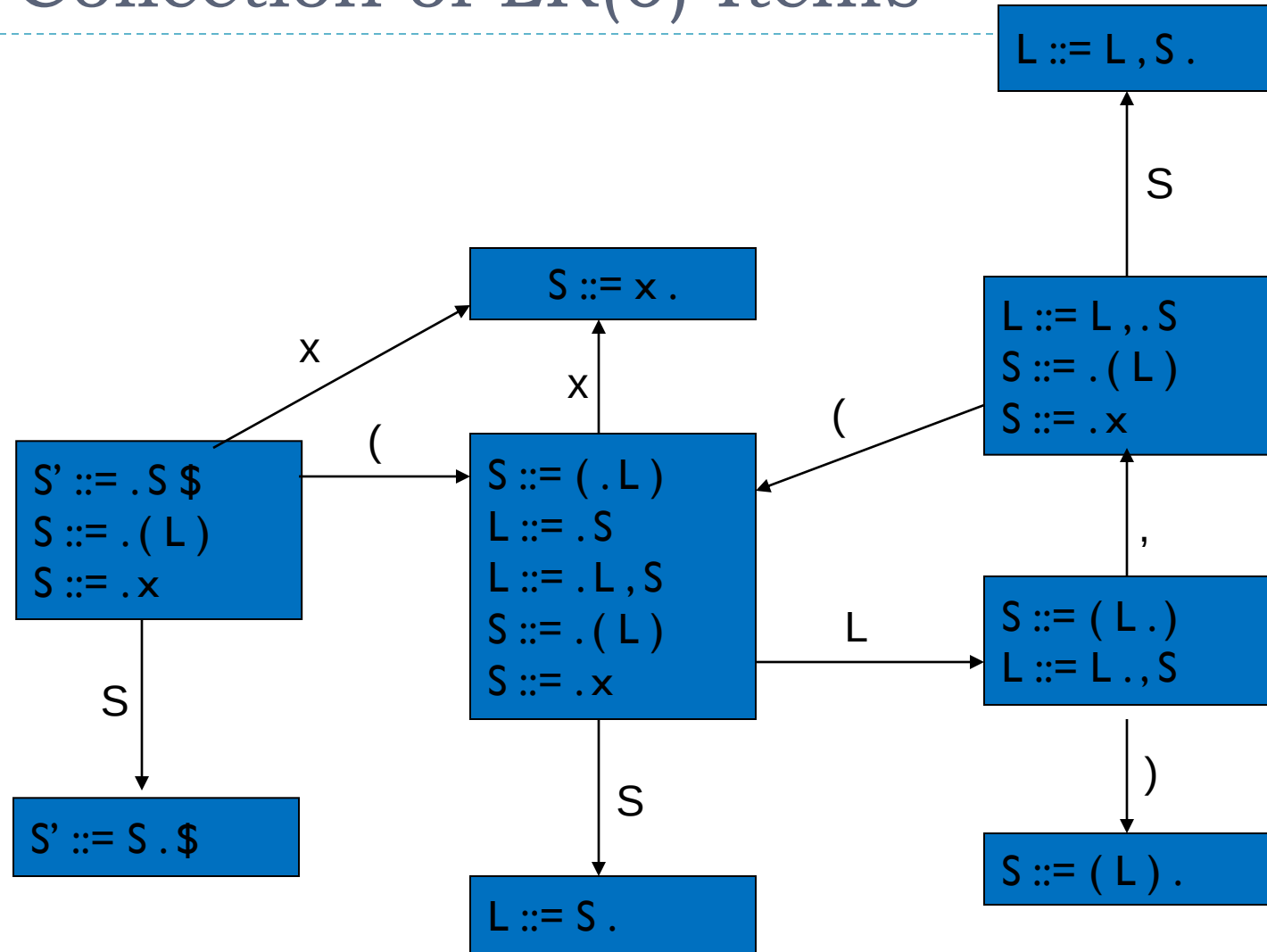
- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L , S$



Canonical Collection of LR(0) Items

Grammar:

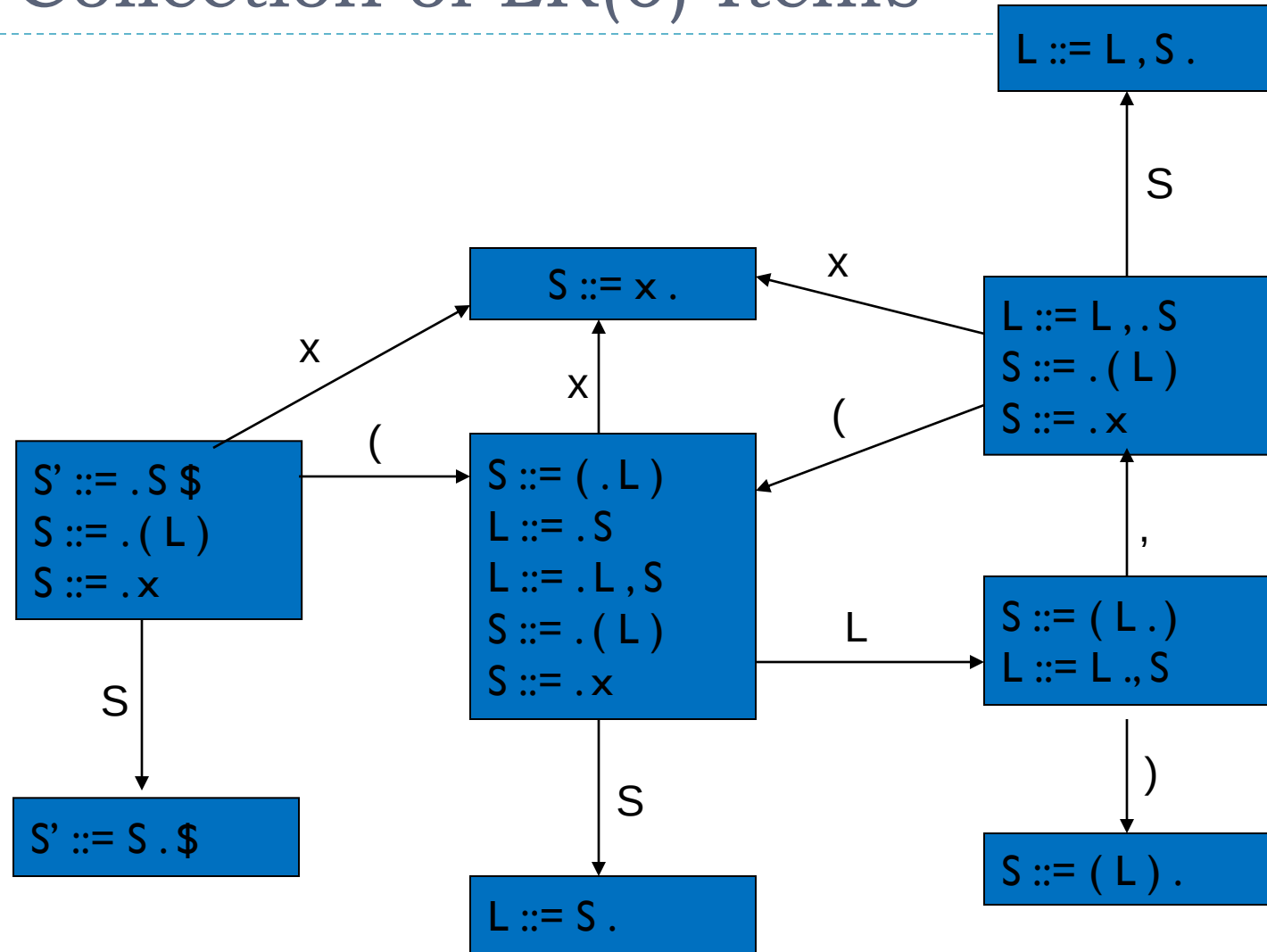
- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L , S$



Canonical Collection of LR(0) Items

Grammar:

- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L , S$

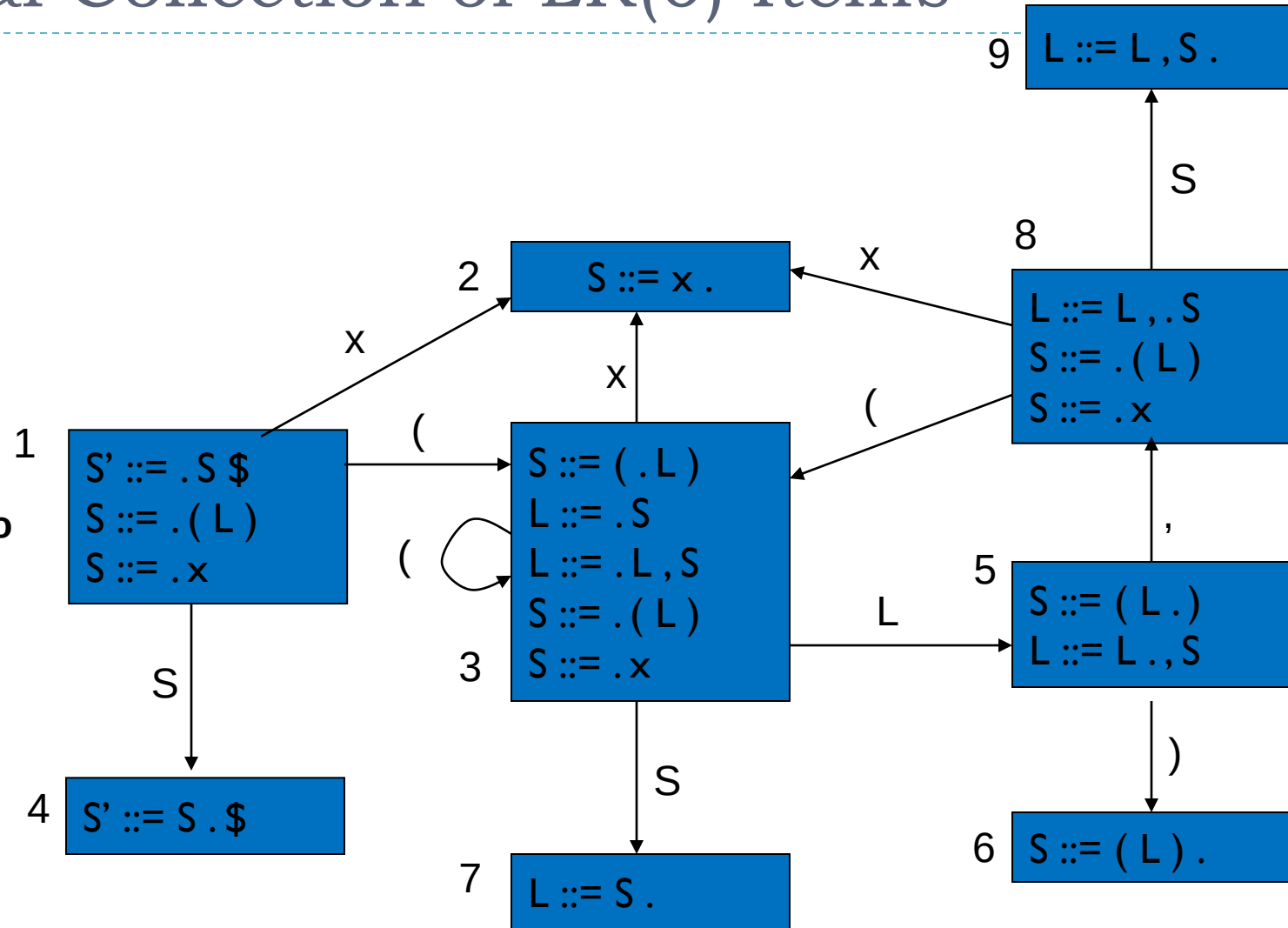


Canonical Collection of LR(0) Items

Grammar:

- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L , S$

Assigning numbers to states:



Computing Parse table

- ▶ At every point in the parse, the LR parser table tells us what to do next according to the automaton state at the top of the stack
 - **shift**
 - **reduce**
 - **accept**
 - **error**

Computing Parse table

- ▶ State i contains $X ::= s . \$ \Rightarrow \text{table}[i, \$] = a$
- ▶ State i contains rule $k: X ::= s . \Rightarrow \text{table}[i, T] = rk$ for all terminals T
- ▶ Transition from i to j marked with $T \Rightarrow \text{table}[i, T] = sj$
- ▶ Transition from i to j marked with $X \Rightarrow \text{table}[i, X] = gj$ for all non-terminals X

states	Terminal seen next ID, NUM, := ...	Non-terminals X,Y,Z ...
1		
2	sn = shift & goto state n	gn = goto state n
3	rk = reduce by rule k	
...	a = accept	
n	= error	

The Parse Table

- ▶ Reducing by rule k is broken into two steps:

- ▶ current stack is:

A 8 B 3 C 7 RHS 12

- ▶ rewrite the stack according to $X ::= \text{RHS}$:

A 8 B 3 C 7 X

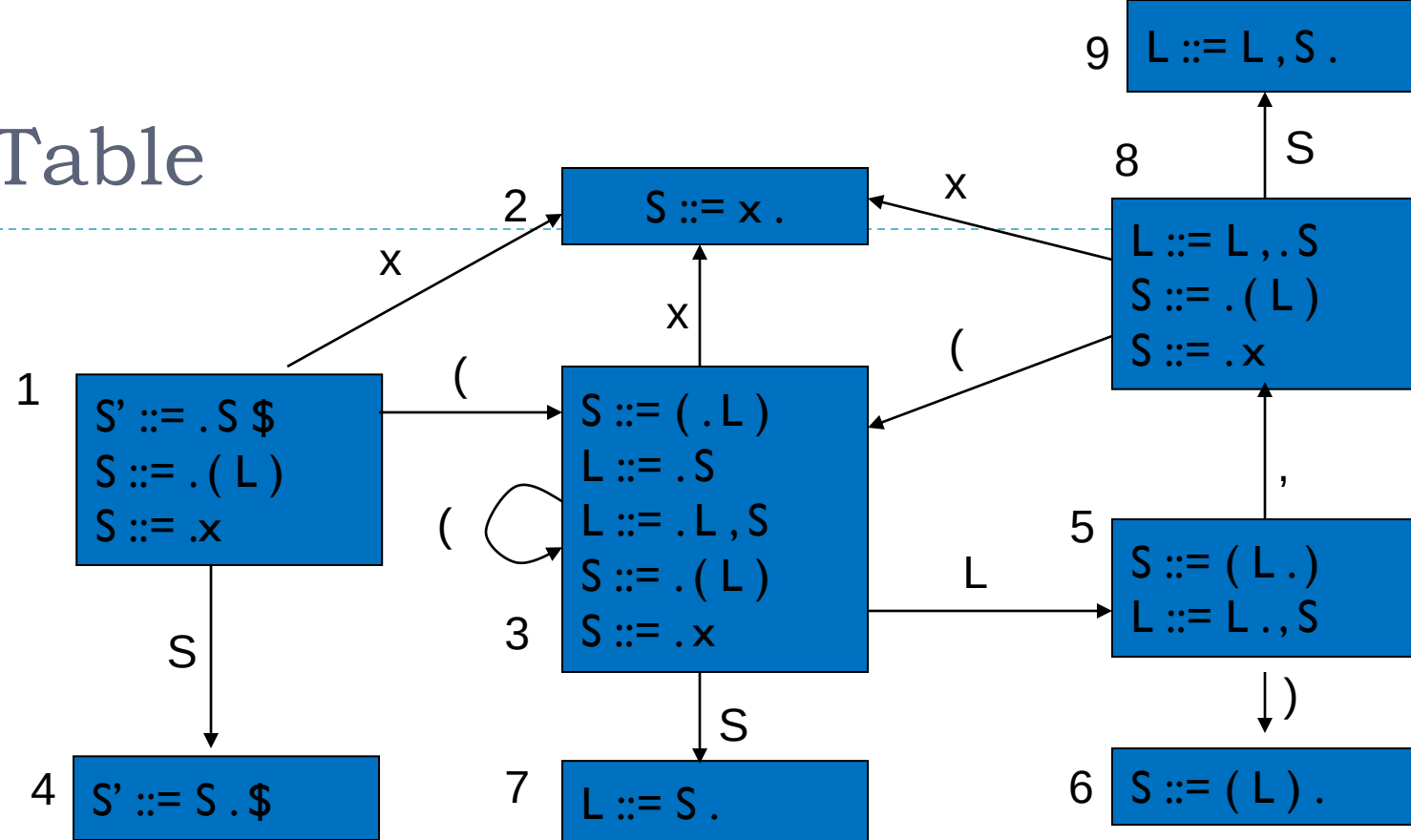
- ▶ figure out state on top of stack (ie: goto 13)

A 8 B 3 C 7 X 13

states	Terminal seen next ID, NUM, := ...	Non-terminals X,Y,Z ...
1		gn = goto state n
2	sn = shift & goto state n	
3	rk = reduce by rule k	
...	a = accept	
n	= error	

The Parse Table

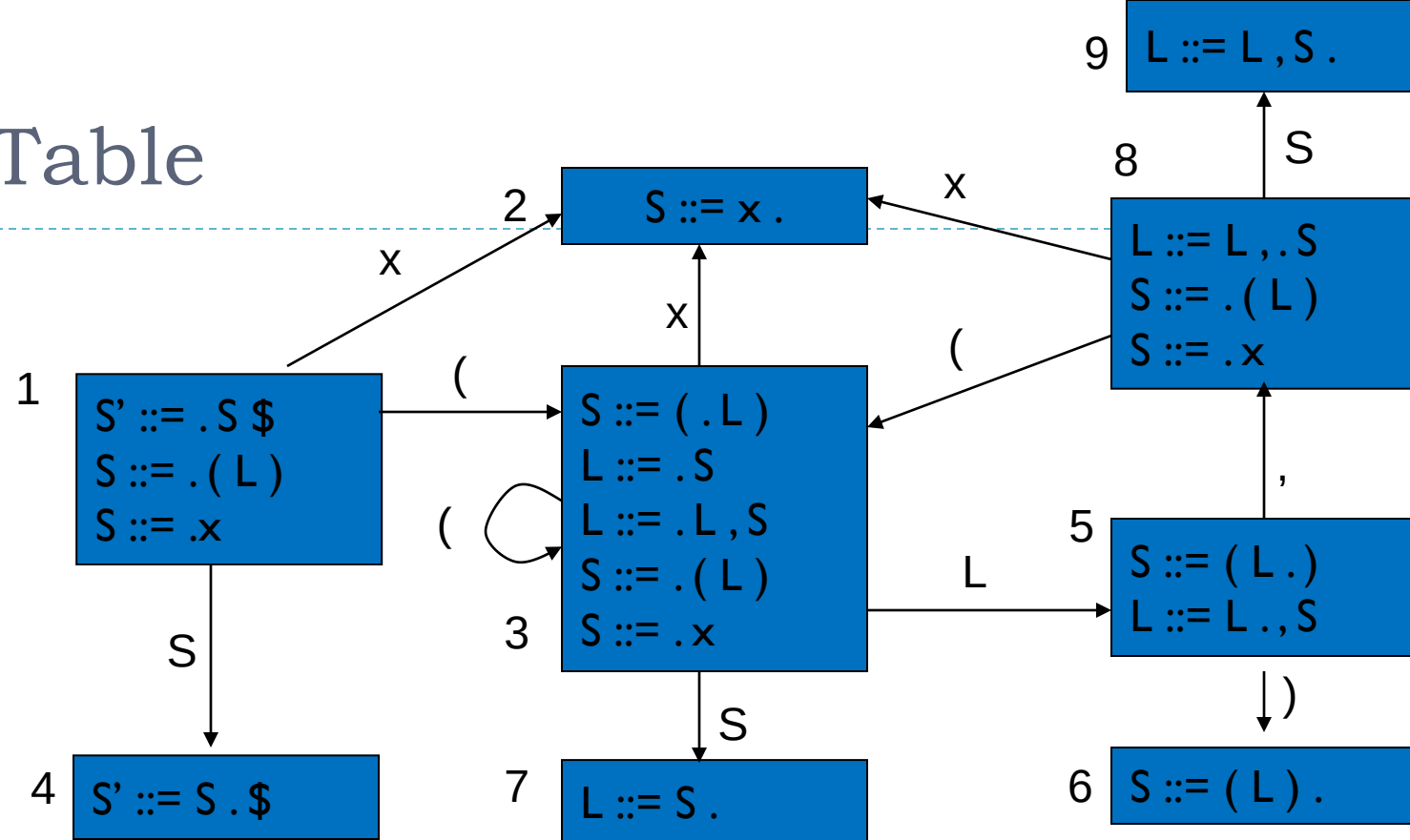
- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L, S$



states	(	)	x	,	\$	S	L
1							
2							
3							
4							
...							

The Parse Table

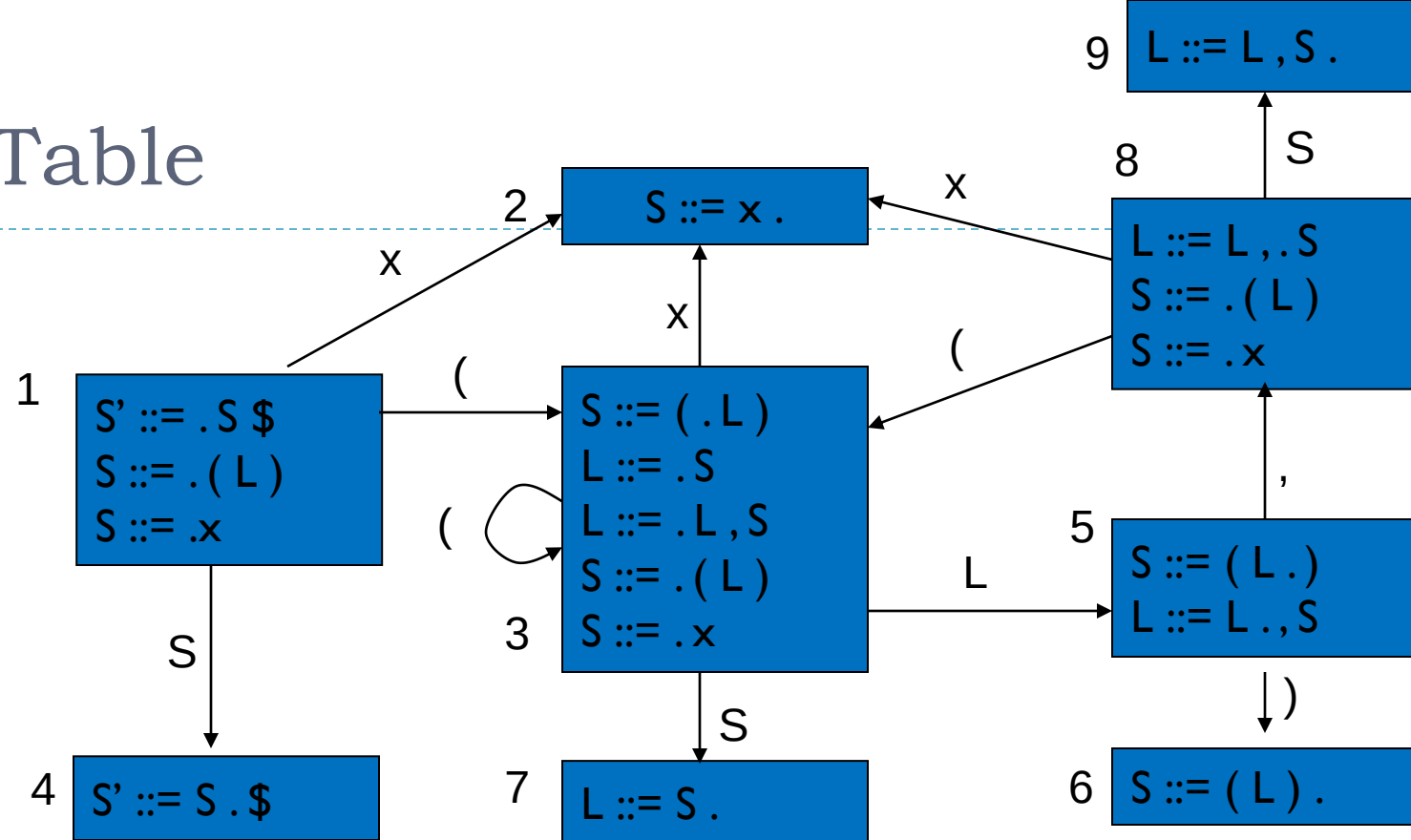
- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L, S$



states	(	)	x	,	\$	S	L
1	s3						
2							
3							
4							
...							

The Parse Table

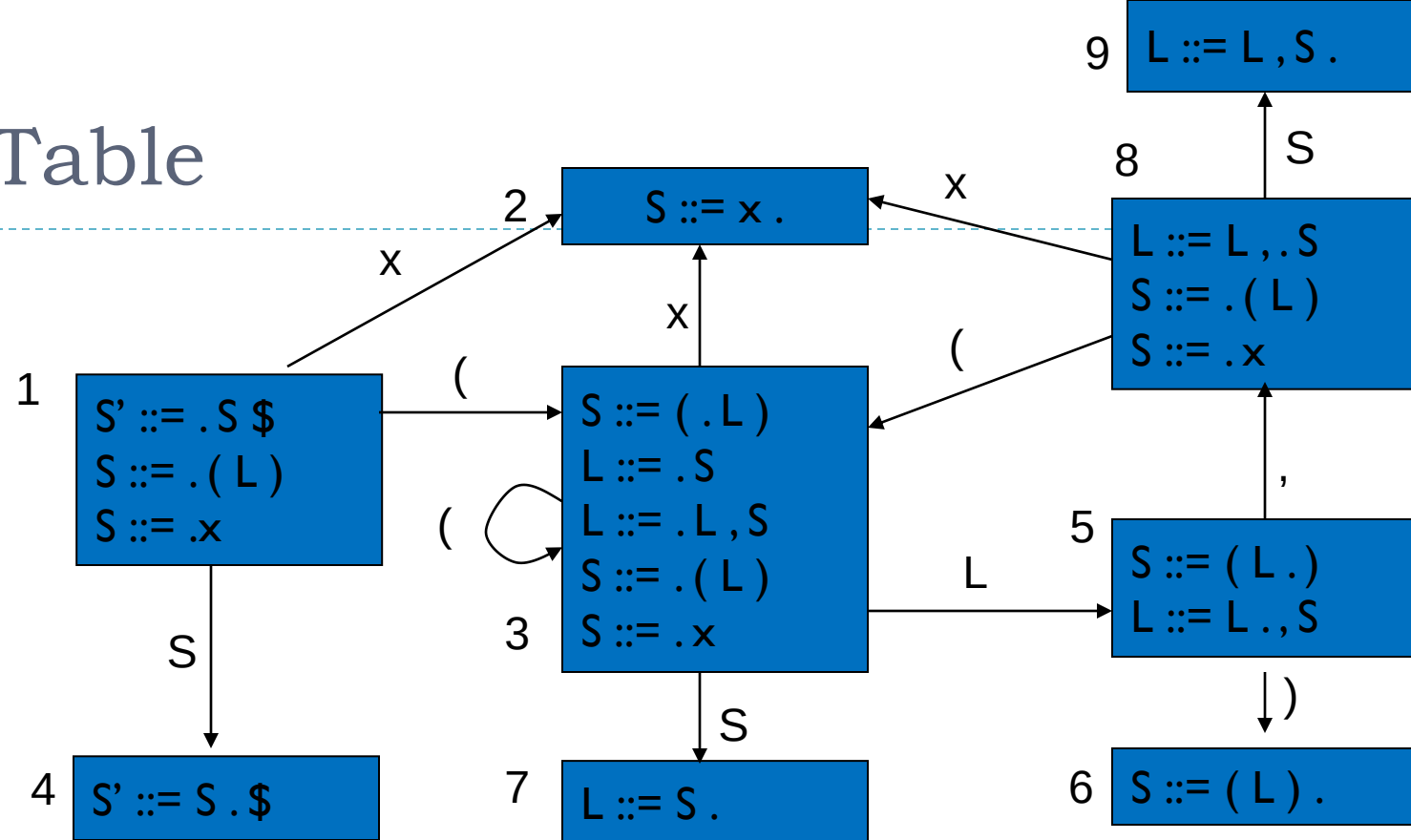
- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L, S$



states	(	)	x	,	\$	S	L
1	s3		s2				
2							
3							
4							
...							

The Parse Table

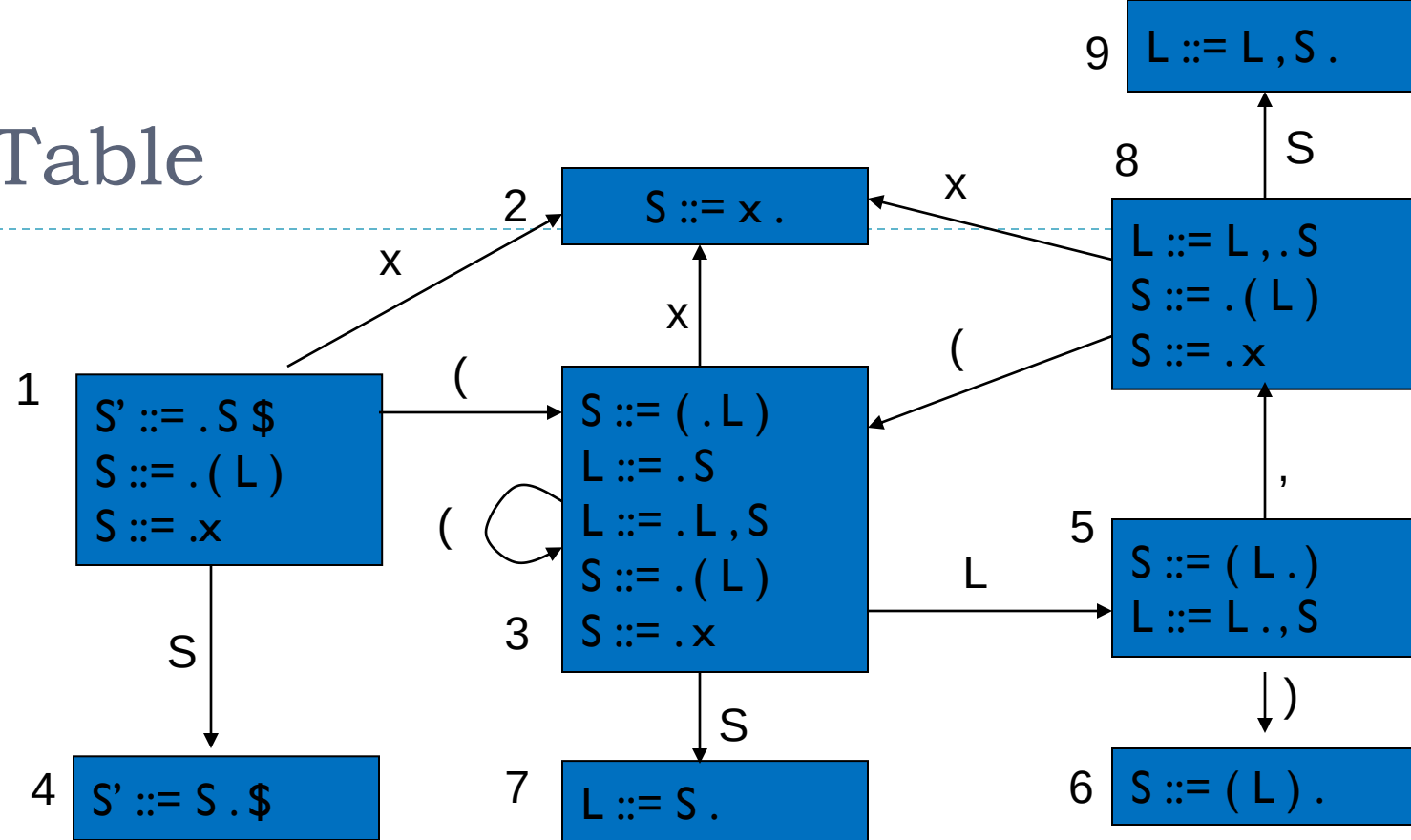
- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L, S$



states	(	)	x	,	\$	S	L
1	s3		s2			g4	
2							
3							
4							
...							

The Parse Table

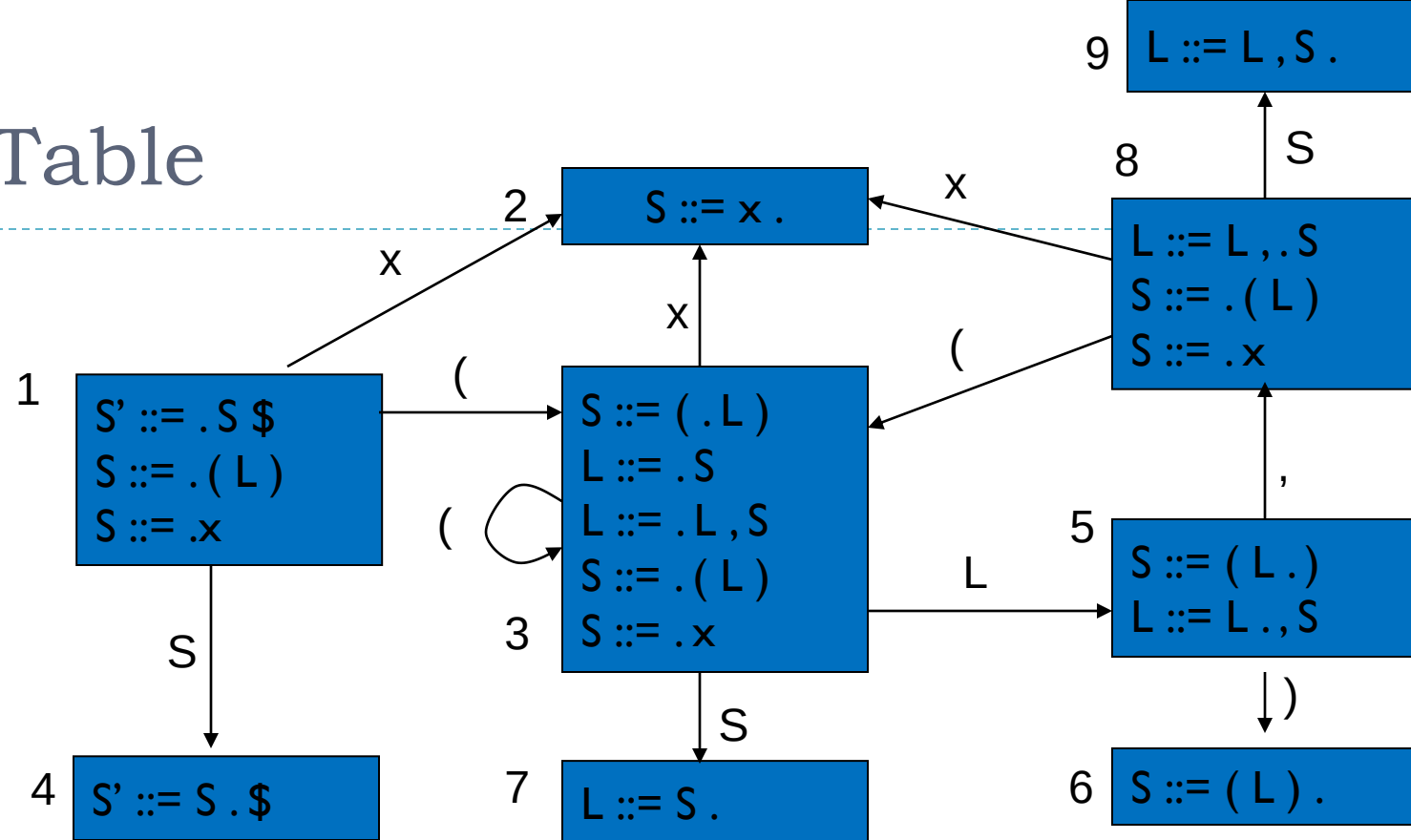
- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L, S$



states	(	)	x	,	\$	S	L
1	s3		s2			g4	
2	r2	r2	r2	r2	r2		
3							
4							
...							

The Parse Table

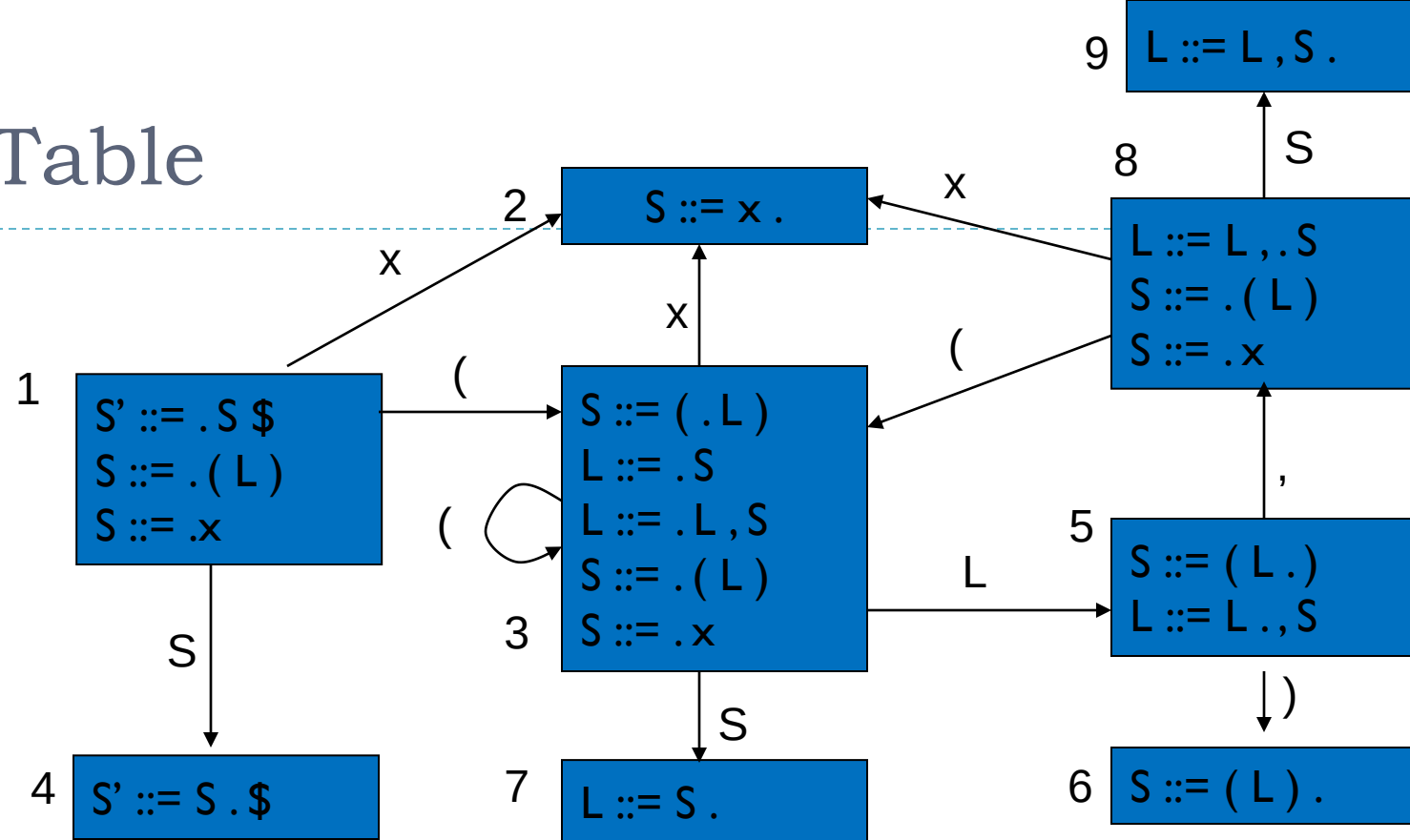
- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L, S$



states	(	)	x	,	\$	S	L
1	s3		s2			g4	
2	r2	r2	r2	r2	r2		
3	s3		s2				
4							
...							

The Parse Table

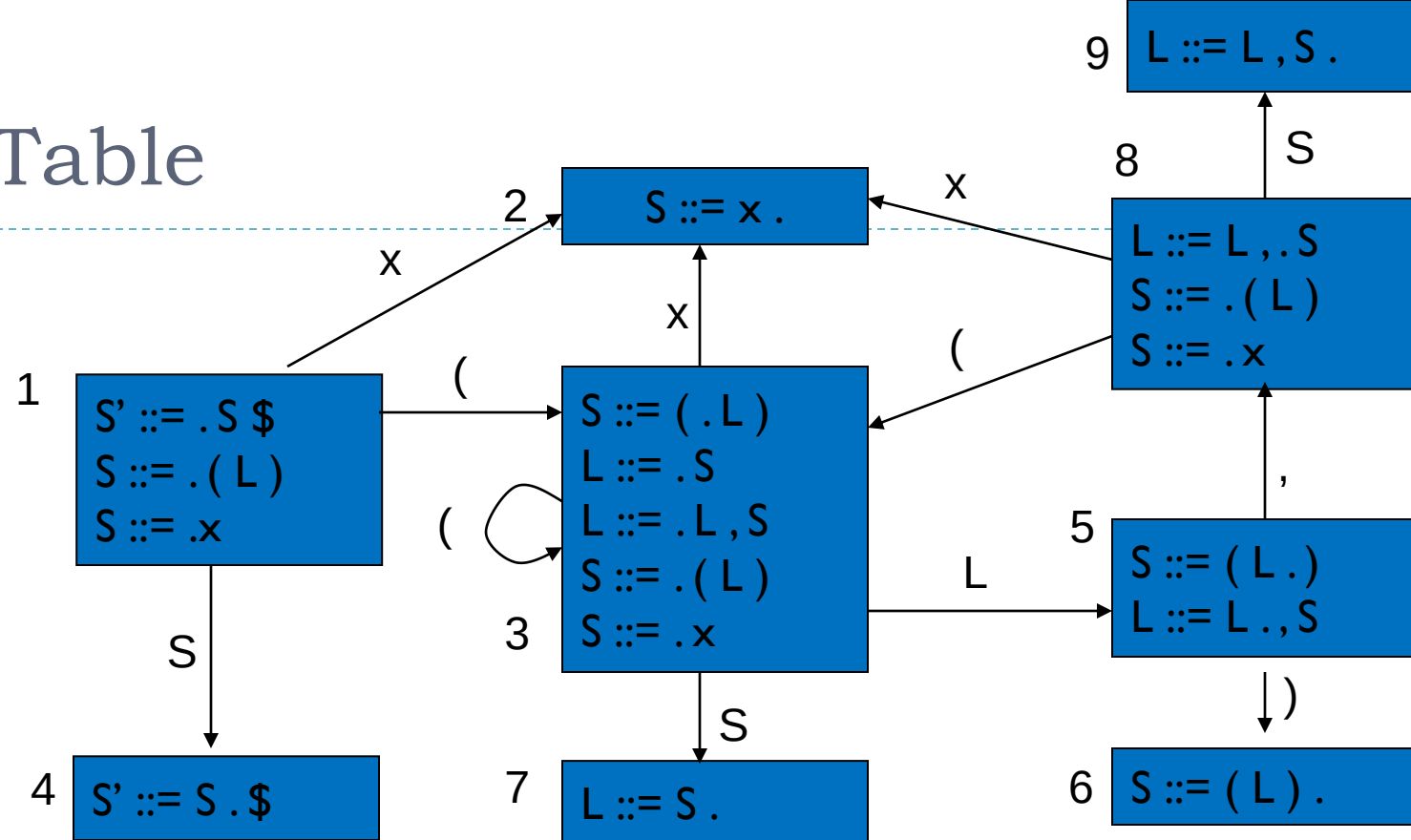
- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L, S$



states	(	)	x	,	\$	S	L
1	s3		s2			g4	
2	r2	r2	r2	r2	r2		
3	s3		s2			g7	g5
4							
...							

The Parse Table

- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L, S$



states	(	)	x	,	\$	S	L
1	s3		s2			g4	
2	r2	r2	r2	r2	r2		
3	s3		s2			g7	g5
4					a		
...							

LR(0) Parsing (Example)

- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L , S$

input: (x , x) \$
stack: 1

yet to read

states	(	)	x	,	\$	S	L
1	s3		s2			g4	
2	r2	r2	r2	r2	r2		
3	s3		s2			g7	g5
4					a		
5		s6		s8			
6	r1	r1	r1	r1	r1		
7	r3	r3	r3	r3	r3		
8	s3		s2			g9	
9	r4	r4	r4	r4	r4		

LR(0) Parsing (Example)

- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L , S$

input: (x , x) \$

stack: 1 (3

yet to read

states	(	)	x	,	\$	S	L
1	s3		s2			g4	
2	r2	r2	r2	r2	r2		
3	s3		s2			g7	g5
4					a		
5		s6		s8			
6	r1	r1	r1	r1	r1		
7	r3	r3	r3	r3	r3		
8	s3		s2			g9	
9	r4	r4	r4	r4	r4		

LR(0) Parsing (Example)

- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L , S$

input: (x , x) \$
 yet to read

stack: 1 (3 x 2

states	(	)	x	,	\$	S	L
1	s3		s2			g4	
2	r2	r2	r2	r2	r2		
3	s3		s2			g7	g5
4					a		
5		s6		s8			
6	r1	r1	r1	r1	r1		
7	r3	r3	r3	r3	r3		
8	s3		s2			g9	
9	r4	r4	r4	r4	r4		

LR(0) Parsing (Example)

- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L , S$

input: (x , x) \$
 yet to read

stack: 1 (3 S

states	(	)	x	,	\$	S	L
1	s3		s2			g4	
2	r2	r2	r2	r2	r2		
3	s3		s2			g7	g5
4					a		
5		s6		s8			
6	r1	r1	r1	r1	r1		
7	r3	r3	r3	r3	r3		
8	s3		s2			g9	
9	r4	r4	r4	r4	r4		

LR(0) Parsing (Example)

- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L , S$

input: (x , x) \$
 yet to read

stack: 1 (3 S 7

states	(	)	x	,	\$	S	L
1	s3		s2			g4	
2	r2	r2	r2	r2	r2		
3	s3		s2			g7	g5
4					a		
5		s6		s8			
6	r1	r1	r1	r1	r1		
7	r3	r3	r3	r3	r3		
8	s3		s2			g9	
9	r4	r4	r4	r4	r4		

LR(0) Parsing (Example)

- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L , S$

input: (x , x) \$
 yet to read

stack: 1 (3 L

states	(	)	x	,	\$	S	L
1	s3		s2			g4	
2	r2	r2	r2	r2	r2		
3	s3		s2			g7	g5
4					a		
5		s6		s8			
6	r1	r1	r1	r1	r1		
7	r3	r3	r3	r3	r3		
8	s3		s2			g9	
9	r4	r4	r4	r4	r4		

LR(0) Parsing (Example)

- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L , S$

input: (x , x) \$
 yet to read

stack: 1 (3 L 5

states	(	)	x	,	\$	S	L
1	s3		s2			g4	
2	r2	r2	r2	r2	r2		
3	s3		s2			g7	g5
4					a		
5		s6		s8			
6	r1	r1	r1	r1	r1		
7	r3	r3	r3	r3	r3		
8	s3		s2			g9	
9	r4	r4	r4	r4	r4		

LR(0) Parsing (Example)

- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L , S$

input: (x , x) \$
 yet to read

stack: 1 (3 L 5 , 8

states	(	)	x	,	\$	S	L
1	s3		s2			g4	
2	r2	r2	r2	r2	r2		
3	s3		s2			g7	g5
4					a		
5		s6		s8			
6	r1	r1	r1	r1	r1		
7	r3	r3	r3	r3	r3		
8	s3		s2			g9	
9	r4	r4	r4	r4	r4		

LR(0) Parsing (Example)

- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L , S$

input: (x , x) \$
 yet to read

stack: 1 (3 L 5 , 8 x 2

states	(	)	x	,	\$	S	L
1	s3		s2			g4	
2	r2	r2	r2	r2	r2		
3	s3		s2			g7	g5
4					a		
5		s6		s8			
6	r1	r1	r1	r1	r1		
7	r3	r3	r3	r3	r3		
8	s3		s2			g9	
9	r4	r4	r4	r4	r4		

LR(0) Parsing (Example)

- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L , S$

input: (x , x) \$
 yet to read

stack: 1 (3 L 5 , 8 S

states	(	)	x	,	\$	S	L
1	s3		s2			g4	
2	r2	r2	r2	r2	r2		
3	s3		s2			g7	g5
4					a		
5		s6		s8			
6	r1	r1	r1	r1	r1		
7	r3	r3	r3	r3	r3		
8	s3		s2			g9	
9	r4	r4	r4	r4	r4		

LR(0) Parsing (Example)

- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L , S$

input: (x , x) \$
 yet to read

stack: 1 (3 L 5 , 8 S 9

states	(	)	x	,	\$	S	L
1	s3		s2			g4	
2	r2	r2	r2	r2	r2		
3	s3		s2			g7	g5
4					a		
5		s6		s8			
6	r1	r1	r1	r1	r1		
7	r3	r3	r3	r3	r3		
8	s3		s2			g9	
9	r4	r4	r4	r4	r4		

LR(0) Parsing (Example)

- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L , S$

input: (x , x) \$
 yet to read

stack: 1 (3 L

states	(	)	x	,	\$	S	L
1	s3		s2			g4	
2	r2	r2	r2	r2	r2		
3	s3		s2			g7	g5
4					a		
5		s6		s8			
6	r1	r1	r1	r1	r1		
7	r3	r3	r3	r3	r3		
8	s3		s2			g9	
9	r4	r4	r4	r4	r4		

LR(0) Parsing (Example)

- $S' ::= S \$$
- $S ::= (L)$
- $S ::= x$
- $L ::= S$
- $L ::= L , S$

input: (x , x) \$
 yet to read

stack: 1 (3 L 5 etc

states	(	)	x	,	\$	S	L
1	s3		s2			g4	
2	r2	r2	r2	r2	r2		
3	s3		s2			g7	g5
4					a		
5		s6		s8			
6	r1	r1	r1	r1	r1		
7	r3	r3	r3	r3	r3		
8	s3		s2			g9	
9	r4	r4	r4	r4	r4		

LR(0) Parsing (Example)

- ▶ Even though we are doing LR(0) parsing we are using some look ahead (there is a column for each non-terminal)
- ▶ however, we only use the terminal to figure out which state to go to next, not to decide whether to shift or reduce

states	(	)	x	,	\$	S	L
1	s3		s2			g4	
2	r2	r2	r2	r2	r2		
3	s3		s2			g7	g5

LR(0) Conflicts

- ▶ Even though we are doing LR(0) parsing we are using some look ahead (there is a column for each non-terminal)
- ▶ however, we only use the terminal to figure out which state to go to next, not to decide whether to shift or reduce

states	(	)	x	,	\$	S	L
1	s3		s2			g4	
2	r2	r2	r2	r2	r2		
3	s3		s2			g7	g5



ignore next automaton state

states	no look-ahead	S	L
1	shift	g4	
2	reduce 2		
3	shift	g7	g5

LR(0) Conflicts

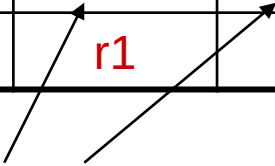
- ▶ Even though we are doing LR(0) parsing we are using some look ahead (there is a column for each non-terminal)
- ▶ however, we only use the terminal to figure out which state to go to next, not to decide whether to shift or reduce
- ▶ If the same row contains both shift and reduce, we will have a conflict ==> the grammar is not LR(0)
- ▶ Likewise if the same row contains reduce by two different rules

states	no look-ahead	S	L
1	shift, reduce 5	g4	
2	reduce 2, reduce 7		
3	shift	g7	g5

SLR Parser

- ▶ SLR (simple LR) is a variant of LR(0) that reduces the number of conflicts in LR(0) tables by using a tiny bit of look ahead
- ▶ To determine when to reduce, **1 symbol of look ahead** is used.
- ▶ **Only put reduce by rule ($X ::= \text{RHS}$) in column T if T is in Follow(X)**

states	(	)	x	,	\$	S	L
1	s3		s2			g4	
2	r2	s5	r2				
3	r1		r1	r5	r5	g7	g5



cuts down the number of rk slots & therefore cuts down conflicts

Assignment

► **Given Grammar:**

$S \rightarrow (L) \mid a$

$L \rightarrow L,S \mid s$

and Input String: (a,a,a)

Compute LR(0) Parsing Table and show the elements of the stack for the given input string.